

Efficient and Fair Computing Power Provision for Edge Systems: A Cooperative Bargaining Approach

Beiyan Liu^a, Yifei Wu^b, Guocheng Liao^{a,*}, Jianguo Chen^a, Qian Ma^b and Xu Chen^c

^a*School of Software Engineering, Sun Yat-sen University, Zhuhai, 519082, China*

^b*School of Intelligent Systems Engineering, Sun Yat-sen University, Shenzhen, 518107, China*

^c*School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, 510275, China*

ARTICLE INFO

Keywords:

Mobile Edge Computing
Computing Power Provision
Nash Bargaining
Social Welfare Maximization
Task Dispatch
Reward Distribution
ADMM

ABSTRACT

Mobile Edge Computing (MEC) enables Edge Service Consumers (ESCs) to offload their computation-intensive, delay-sensitive tasks to an Edge Service Provider (ESP). However, the ESP's fixed, self-owned edge servers often lack the flexibility to accommodate elevated demand levels during peak periods without costly over-provisioning. To address this, we propose a cooperative computing power provision framework for edge systems, where the ESP collaborates with nearby Mobile Devices (MDs) to leverage their under-utilized computing resources. A key issue in such a framework is incentivizing the self-interested MDs to cooperate. In this paper, we propose a Nash bargaining solution-based mechanism to enhance the cooperation between the ESP and the MDs, formulating the joint optimization of task dispatch, computing power allocation, and reward distribution as a Nash Bargaining Problem (NBP). By observing the efficiency–fairness separation structure in NBP, we first decompose the NBP into a convex Social Welfare Maximization (SWM) problem and an Optimal Reward Distribution (ORD) problem. Then, we design two distributed algorithms to solve these subproblems: an Alternating Direction Method of Multipliers (ADMM)-based algorithm for the SWM problem and a negotiation algorithm for the ORD problem. Crucially, both algorithms require limited private-parameter exchange. Extensive experiments show that our proposed method achieves the highest social welfare with competitive delay under real-world dynamic task arrivals and MD mobility. Further results demonstrate improved fairness in utility and task-dispatch, and effective scalability to 1,000 MDs.

1. Introduction

The rapid proliferation of smart devices has brought the popularity of many intelligent applications (e.g., augmented reality [31] and face recognition [65]), which demand low application delay and intensive computation. Driven by these intelligent applications, Mobile Edge Computing (MEC) has emerged as a pivotal architectural paradigm that pushes computation from remote clouds to the network edge for computation-intensive and delay-sensitive services [1].

Under the MEC paradigm, an Edge Service Provider (ESP) offers edge computing services to a crowd of Edge Service Consumers (ESCs), who offload their tasks to the ESP. These tasks are delay-sensitive and require timely processing to ensure the Quality of Service (QoS) guarantees. The ESP can typically process the offloaded tasks via its own edge servers (e.g., [54, 8]). This has direct resource control but limits flexibility and scalability, particularly when handling peak-hour demand surges. For example, during peak traffic periods, a large number of connected vehicles and roadside cameras may generate real-time object-detection and traffic-event analysis tasks, raising the aggregate computation demand substantially above its normal level. An edge-server cluster provisioned for normal demand level may then

become overloaded, whereas permanently provisioning it for the peak level would leave substantial resources idle during off-peak periods.

Alternatively, the ESP can elastically recruit nearby Mobile Devices (MDs) to complement its fixed computing capacity. Here, MDs broadly refer to user-owned or third-party-owned devices, such as smartphones, tablets, laptops, and other nearby edge devices, that can contribute part of their under-utilized computing resources. Modern consumer-grade MDs are increasingly capable of supporting computation-intensive workloads. For example, Apple's M4 chip offers up to 38 TOPS of computing power [4], while the NVIDIA GeForce RTX 5080 laptop GPU delivers over 1,300 TOPS for AI-centric workloads [39]. These advances make MDs promising auxiliary computing nodes for provisioning additional capacity during temporary high-demand periods, without requiring the ESP to maintain costly peak-provisioned infrastructure.

Such device-level cooperative computing markets have become a reality. Several commercial platforms have already demonstrated the feasibility of utilizing MD computing resources. For example, Theta Edge Node enables users to earn rewards by running AI computation tasks directly on their devices [50]. SaladCloud builds a distributed GPU cloud by leveraging individual compute nodes and rewarding device owners for sharing idle resources [47]. Render Network provides a peer-to-peer marketplace where GPU owners monetize idle or under-utilized GPU computing power [44]. PPIO pools distributed computing resources to provide

*Corresponding author.

liuby57@mail2.sysu.edu.cn (B. Liu); wuyf255@mail2.sysu.edu.cn (Y. Wu); liaogch6@mail.sysu.edu.cn (G. Liao); chenjc33@mail.sysu.edu.cn (J. Chen); maqian25@mail.sysu.edu.cn (Q. Ma); chenxu35@mail.sysu.edu.cn (X. Chen)

computing-power, model, and edge-computing services for AI, audio, video, and metaverse applications [41]. These real-world examples indicate that the cooperative computing market is becoming commercially viable. In such systems, the MDs execute the assigned tasks, while the economic reward is credited to the MD owner.

The above observations reveal both an opportunity and a challenge. The opportunity is that nearby MDs can expand the ESP's computing capacity during demand surges, and improve the QoS perceived by the ESCs. The challenge is that these MDs are self-interested. Contributing computing power consumes energy and reduces the resources available for the MDs' own local applications. Hence, an MD may refuse to participate unless the cooperation brings it sufficient benefit compared with non-cooperation. Therefore, a practical cooperative computing-power provision framework must address two tightly coupled questions:

Key Question 1 (Efficiency): *How can the ESP and the MDs cooperate and maximize social welfare?*

Key Question 2 (Fairness): *How to fairly distribute reward so that self-interested MDs are willing to contribute?*

Existing MEC task-offloading and resource-allocation studies (e.g., [34, 16, 67, 32, 23, 3, 18]) often assume that MDs are willing to contribute resources and ignore the incentive issue of self-interested MDs. Meanwhile, incentive mechanisms in MEC (e.g., [42, 14, 35, 66, 26, 25, 28]) mainly focus on designing market rules, not modeling the ESP and the MDs as cooperative participants who jointly create and share a cooperation surplus. Hence, how to jointly achieve efficiency and fairness in cooperative computing-power provision with self-interested MDs remains underexplored.

This gap motivates us to adopt the Nash Bargaining Solution (NBS) [37, 5, 40] to incentivize ESP–MD cooperation. The NBS naturally addresses the two key questions of the cooperative computing-power provision framework. From the efficiency perspective, the NBS maximizes the social welfare between the ESP and the MDs, thereby maximizing the cooperation surplus created by the ESP–MD cooperation. From the fairness perspective, it guarantees individual rationality, ensuring that both the ESP and the MDs obtain utilities no lower than their disagreement points. It also provides proportional fairness in sharing the cooperation surplus according to their bargaining powers. Thus, the NBS provides a unified framework for simultaneously achieving efficient computing-power provision and fair reward distribution in the cooperative edge systems.

In this paper, we present an efficient and fair computing power provision framework for edge systems. The ESP dispatches tasks to the MDs and distributes rewards, while the MDs provide computing power for the dispatched tasks. We formulate the joint optimization of task dispatch, computing power allocation, and reward distribution as a Nash Bargaining Problem (NBP). We observe an efficiency–fairness separation structure in the NBP. The task dispatch rates and computing-power allocations determine the total cooperation surplus created by the ESP–MD collaboration, while the

rewards determine how this surplus is shared among the ESP and the MDs. Based on this structure, we decompose the NBP into a Social Welfare Maximization (SWM) problem and an Optimal Reward Distribution (ORD) problem. Next, based on the Alternating Direction Method of Multipliers (ADMM) [6], we propose a distributed algorithm to solve the SWM problem. We further design a distributed negotiation algorithm for the ORD problem. Combining the solution to the SWM problem and the solution to the ORD problem yields the solution to the original NBP. We highlight that these two algorithms require limited private-parameter exchange. To sum up, our method enables efficient and fair computing power provision in the edge system.

The main contributions of this paper are as follows:

- **An Efficient and Fair Cooperative Computing Power Provision Framework:** We propose a novel framework for efficient and fair computing power provision in edge systems, where an ESP collaborates with a set of self-interested MDs to serve the ESCs' delay-sensitive computation tasks. The framework ensures system-wide efficiency as it maximizes social welfare, and guarantees fairness through optimal reward distribution.
- **Distributed Algorithms with Limited Private-Parameter Exchange:** We formulate the joint optimization of task dispatch, computing power allocation, and reward distribution as an NBP. Based on the efficiency–fairness separation structure in the NBP, we decompose the original NBP into an SWM problem and an ORD problem. Furthermore, we design an ADMM-based distributed algorithm for the SWM problem and a negotiation algorithm for the ORD problem. Both algorithms require limited private-parameter exchange.
- **Comprehensive Performance Evaluation:** We evaluate the proposed method under dynamic task arrivals and MD mobility using traces from real-world datasets. The proposed method achieves the highest social welfare and maintains a competitive delay using substantially less computing power. Moreover, it achieves higher fairness among the MDs, and ensures a more balanced utility distribution across the ESP and the MDs, compared with non-cooperative baseline.

The remainder of this paper is organized as follows. Section 2 discusses the related works. Section 3 introduces the system model of the cooperative computing power provision framework. Section 4 formulates the NBP and performs problem decomposition. Section 5 presents the distributed ADMM-based algorithm for the SWM problem. Section 6 shows the negotiation algorithm for the ORD problem. Section 7 presents the numerical results. Section 8 concludes the paper.

2. Related Work

In this section, we review some related works regarding MEC task offloading and resource allocation, incentive mechanisms in MEC, and NBS-based allocation methods.

2.1. MEC Task Offloading and Resource Allocation

Recent surveys and studies highlight the practical requirements of edge and fog systems [22, 46, 45]. For example, Jamil *et al.*[22] provide a taxonomy and review of resource allocation and task scheduling in fog and Internet-of-Everything environments. Saedi *et al.*[46] develop a zero-trust rule-based protocol for secure 5G handover against rogue base stations. Rezaeianfardouei *et al.*[45] study IPsec parameter tuning for reducing failover latency in site-to-site VPNs.

Among these practical concerns, one of the most actively studied directions is task offloading and resource allocation in MEC (e.g., [34, 16, 67, 32, 23, 3, 18]). Luo *et al.*[34] consider minimizing the delay and cost of task offloading for vehicular edge computing and propose an algorithm based on particle swarm optimization. Dai *et al.*[16] study the Unmanned Aerial Vehicle (UAV)-assisted vehicular task offloading problem in vehicular edge computing networks and propose an online UAV-assisted task offloading algorithm. Zhu *et al.*[67] study the single-user multi-edge-server MEC system based on downlink non-orthogonal multiple access and propose a deep reinforcement learning-based algorithm. Liu *et al.*[32] design a vehicle-to-vehicle offloading strategy that leverages one-hop or multi-hop vehicles to improve resource utilization and user experience. Jiang *et al.*[23] propose an online joint task offloading and resource allocation framework under the long-term MEC energy constraint, aiming at guaranteeing the end-users' quality of experience. Apostolopoulos *et al.*[3] present a novel user-optimal data offloading strategy in MEC with ground and UAV-mounted servers and formulate the problem as a non-cooperative game. Hao *et al.*[18] study joint task offloading, resource allocation, and trajectory design for multi-UAV cooperative edge computing with task priority and propose a deep reinforcement learning algorithm to solve the problem.

Despite these advances, most of the above works mostly assume that the agents are not self-interested and provide their resources without payment, which is impractical in reality. In this paper, we address the self-interest of the MDs and jointly optimize the ESP's task dispatch and the MDs' computing power allocation, utilizing the NBS to incentivize the cooperation between the ESP and the MDs.

2.2. Incentive Mechanisms in MEC

To motivate agents to participate in the MEC system, extensive studies focus on the incentive mechanisms in MEC, such as auction mechanisms (e.g., [42, 14, 35, 66]), pricing schemes (e.g., [26]), and contract mechanisms (e.g., [25, 28]). Pu *et al.*[42] explore an MEC system that enables collaborative task offloading, where busy users can offload their tasks to the server or idle users, or perform them

locally. They propose a computing resource sharing auction algorithm to incentivize idle users to participate in task offloading and share their computing resources. Chen *et al.*[14] propose a general service-oriented double auction mechanism to tackle the joint problem of incentive design and service provisioning for edge computing. Ma *et al.*[35] propose a truthful combinatorial double auction mechanism with a dynamic pricing strategy for resource allocation in the constrained MEC environment. Zhou *et al.*[66] design a reverse auction-based computation offloading and resource allocation mechanism for mobile cloud-edge computing. Li *et al.*[26] study task offloading in MEC and propose an online nonlinear incentive mechanism with energy budget. Li *et al.*[25] design a contract-based incentive mechanism to encourage parked vehicles to contribute their idle computational resources to the service provider considering parking time and energy consumption. Li *et al.*[28] present a vehicular fog-edge computing network for computation offloading and propose a multistage Stackelberg game and a contract-based incentive mechanism.

The above incentive mechanisms provide useful tools for motivating resource contribution in MEC. However, they view the ESP-MD interaction based on market transactions. Pricing-based and auction-based mechanisms often treat the ESP as a resource buyer or market platform and the MDs as resource sellers or bidders, while contract-based mechanisms usually model them as a principal and agents under information asymmetry. Accordingly, these works mainly focus on designing market rules to induce participation, truthful behavior, or proper self-selection.

In contrast, our work models the ESP and the MDs as cooperative participants who jointly create and fairly share a cooperation surplus. Using an NBS-based formulation, we jointly address efficient task dispatch, computing-power allocation, and fair reward distribution, and design distributed algorithms that require limited private-parameter exchange.

2.3. NBS-based Allocation Methods

The NBS is a widely adopted framework to promote cooperation among self-interested players (e.g., [20, 5, 52, 63, 9, 51, 27]). Iosifidis *et al.*[20] design a general framework for User-Provided Network (UPN) services and propose an NBS-based distributed incentive mechanism to ensure users' participation. Boche *et al.*[5] study strategies satisfying Nash bargaining and proportional fairness for wireless systems. Wang *et al.*[52] investigate how to design an incentive mechanism utilizing the NBS to encourage proactive energy trading and fair benefit sharing. Yu *et al.*[63] apply the NBS to analyze cooperation decisions and payments between a mobile network operator and venue owners in public Wi-Fi deployment. Tsigkari *et al.*[51] use the NBS to jointly design recommendation policies and fairly allocate the resulting cooperation gains between a content provider and a caching network provider. Li *et al.*[27] formulate a one-to-many concurrent bargaining game to determine client selection and payments for communication-efficient federated learning.

Cha *et al.* [9] develop NBS-based strategies for efficient and fair allocation of time-varying vehicular-network resources.

While prior works have mainly focused on scenarios such as UPN, energy trading, and public Wi-Fi deployment, our study targets a fundamentally different context of cooperative computing power provision in edge systems. Our formulation integrates task dispatch, computing power allocation, and reward distribution into a unified bargaining framework. The resulting distributed algorithms require limited private-parameter exchange.

3. System Model

3.1. Framework Description

As shown in Fig. 1, we consider a cooperative computing power provision framework with an ESP, a crowd of ESCs, and a set of nearby MDs $\mathcal{N} \triangleq \{1, 2, \dots, N\}$. The ESP provides edge computing services to the ESCs, who offload their delay-sensitive tasks to the ESP.

The workflow of the cooperative computing power provision framework in each scheduling epoch is as follows. The ESCs first offload their delay-sensitive tasks to the ESP. The ESP then dispatches these arriving tasks to nearby MDs via wireless channels. Upon receiving the dispatched tasks, each MD contributes a portion of its local computing power for processing. After task completion, the ESP rewards the MDs for their contributions and collects revenue from the ESCs for the delivered services.

In the following subsections, we first present the task model, including task arrival, task dispatch and task delay. Then, we introduce the utility models for the ESP and the MDs.

3.2. Task Model

3.2.1. Task Arrival

The ESP receives task-offloading requests from independent ESCs. Although the requests generated by individual ESCs are stochastic, their aggregation at the ESP can be approximated by a Poisson process when no single ESC dominates the total workload [2]. Following this common modeling practice in edge computing [61, 24, 10], we model the aggregate task arrivals at the ESP as a Poisson process with an average task arrival rate λ_0 (tasks/s).¹

3.2.2. Task Dispatch

The ESP dispatches λ_n tasks to MD $n \in \mathcal{N}$ per second. Let $\lambda \triangleq [\lambda_1, \lambda_2, \dots, \lambda_N]$ denote the vector of task dispatch rates to the MDs. As all the arriving tasks should be dispatched, naturally we have:

$$\lambda_0 = \sum_{n \in \mathcal{N}} \lambda_n. \quad (1)$$

¹The proposed cooperative computing-power provision framework is not tied to a specific task-arrival model because the NBS mechanism does not rely on a particular task arrival process. We further evaluate the proposed framework under a real-world task-arrival trace in Section 7.2.

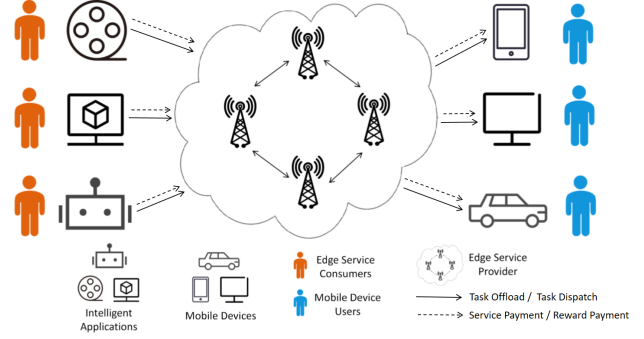


Figure 1: Cooperative computing-power provision framework among ESCs, the ESP, and MD users. ESCs offload delay-sensitive tasks to the ESP, which dispatches the tasks to nearby MDs. Participating MDs allocate local computing power to process the dispatched tasks. After task completion, the ESP collects service payments from ESCs and distributes personalized monetary rewards to the corresponding MD users.

3.2.3. Task Delay

The delay of each dispatched task consists of the transmission delay between the ESP and the MD, and the processing delay at the MD.

Transmission Delay. The ESP transmits a task to the corresponding MD via a wireless channel. Here, we adopt the *Orthogonal Frequency Division Multiple Access (OFDMA)* model similar to previous works [64, 55]. Let B denote the channel bandwidth between the ESP and each MD. The transmission rate from the ESP to MD $n \in \mathcal{N}$ can be calculated using the Shannon formula:

$$R_n = B \log_2 \left(1 + \frac{\xi g_n}{B \sigma_n} \right), \quad (2)$$

where ξ denotes the transmission power of the ESP, g_n denotes the channel power gain, and σ_n denotes the noise power spectral density at the receiver of the MD n . We denote the expected data size of a task as s (in bits). The average transmission delay for a task dispatched to the MD n is:

$$T_n^t = \frac{s}{R_n}. \quad (3)$$

We assume that the transmission delay of the computation result is negligible [13, 12], because the size of the output is significantly smaller than the size of the input data for many applications (e.g., image classification and face recognition).²

Processing Delay. Each MD provides one logical computing service unit to the ESP, where the dispatched tasks wait in a single local queue and are processed sequentially [68, 30, 58, 48]. The tasks dispatched to MD n are modeled as a Poisson arrival process with rate λ_n . Based on this abstraction, we model the task processing at each MD as

²A non-negligible delay in returning the computation result does not affect the proposed NBS framework. An MD-specific return-delay term can be incorporated into D_n without changing the problem structure.

an M/M/1 queue to facilitate tractable delay analysis.³ Each MD $n \in \mathcal{N}$ allocates p_n (in GHz) of local computing power to process the dispatched tasks. Let $\mathbf{p} \triangleq [p_1, \dots, p_N]$ denote the vector of allocated computing powers of all MDs. Let F_n denote the MD n 's computation capacity. The allocated computing power should not exceed the MD n 's capacity, i.e., $p_n \leq F_n$. Let l denote the expected computation intensity of each task, i.e., the expected number of CPU cycles to process one bit of data. The task processing rate of the MD n is p_n/sl (in tasks per second). Therefore, the average processing delay for a task at MD n can be computed using the M/M/1 queuing theory [49]:

$$T_n^c(\lambda_n, p_n) = \frac{1}{p_n/sl - \lambda_n}. \quad (4)$$

To ensure the stability of the queue at the MD n , it is required that $p_n/sl > \lambda_n$.

To sum up, we can derive the expected total delay of a task dispatched to MD $n \in \mathcal{N}$ as follows:

$$D_n(\lambda_n, p_n) = T_n^t + T_n^c(\lambda_n, p_n). \quad (5)$$

3.3. Utility Model

In this subsection, we introduce the utility models for the ESP and the MDs. The ESP's utility accounts for both its revenue from the ESCs and reward payments to the MDs. Meanwhile, each MD's utility is composed of its received reward, energy consumption cost, and valuation of its remaining computing power.

3.3.1. ESP's Utility

The ESP earns revenue from the ESCs by completing their tasks, and in turn rewards the MDs for contributing their local computing power.

QoS. We measure the QoS by the maximum (i.e., the worst-case) task delay across all MDs, which is defined as $D_{\max} \triangleq \max_{n \in \mathcal{N}} \{D_n\}$. A smaller $D_{\max}$ indicates a higher QoS, since the ESP can guarantee a lower worst-case delay for all ESCs' offloaded tasks. If even one MD fails to satisfy the delay requirement, the resulting worst-case delay degrades the service experience and may cause revenue loss for the ESP. We consider a delay tolerance threshold D_0 of the ESCs, i.e., the maximum delay that the ESCs can accept. Thus, we impose the constraint $D_{\max} < D_0$. If this threshold is violated, the delivered service is regarded as unacceptable for the considered delay-sensitive applications, and no valid service revenue is generated.

ESP's Revenue. We model the ESP's revenue from ESCs as a concave decreasing function $Q(D_{\max})$ of the worst-case delay $D_{\max}$. It captures two key economic effects. First, $Q(D_{\max})$ is decreasing in $D_{\max}$, since a larger worst-case delay leads to poorer QoS and lower revenue. Second,

$Q(D_{\max})$ is concave in $D_{\max}$. This means that the revenue loss becomes increasingly severe as $D_{\max}$ approaches the delay tolerance threshold D_0 , while reducing an already small delay yields a smaller marginal revenue improvement. The strict constraint $D_{\max} < D_0$ ensures that the revenue function is well-defined within the feasible QoS region.

The Rewards Offered by ESP to MDs. The ESP rewards the MDs for their local computing power provision. We adopt a discriminatory rewarding scheme similar to previous works [11, 60]. The ESP pays a personalized reward r_n to the user of each MD $n \in \mathcal{N}$. Let $\mathbf{r} \triangleq [r_1, \dots, r_N]$ denote the vector of rewards offered to MDs.

The ESP's Utility. The ESP's utility is the revenue $Q(D_{\max})$ minus the total rewards paid to the MDs:

$$U_0 = Q(D_{\max}) - \sum_{n \in \mathcal{N}} r_n. \quad (6)$$

3.3.2. MD's Utility

Each MD $n \in \mathcal{N}$ receives a reward r_n from the ESP for providing its computing power. Meanwhile, its utility also includes the energy consumption incurred by computation, and its valuation of the remaining computing power.

Energy Consumption Cost. The MD n consumes energy when providing computing power p_n to process tasks. We consider a widely adopted quadratic energy consumption model [56, 36] as follows:

$$E_n(p_n) = c_n p_n^2, \quad (7)$$

where c_n is the energy consumption coefficient that characterizes the hardware-specific power efficiency of the MD n .

Valuation of Remaining Computing Power. Since each MD has its own computational needs (e.g., online gaming, video streaming), providing computing power to the ESP reduces the MD's resources to serve that need. We consider that each MD has a valuation of its remaining computing power. Recall that F_n denotes the MD n 's computation capacity. If the MD n allocates p_n units of computing power for the dispatched tasks, it leaves itself $F_n - p_n$ for its own usage. Its valuation of the remaining computing power is modeled by a concave function [59]:

$$b_n(p_n) = (F_n - p_n)^{k_n}, \quad (8)$$

where $k_n \in (0, 1)$ captures the MD n 's sensitivity of remaining computing power. A larger k_n indicates that the MD focuses more on meeting its own needs rather than offering computing power to the ESP.

Finally, the MD n 's utility is

$$U_n = r_n + b_n(p_n) - E_n(p_n). \quad (9)$$

In summary, the cooperative computing power provision framework involves three decision variables: the ESP decides the task dispatch rates λ and the corresponding rewards $\mathbf{r}$ for the MDs, while each MD $n \in \mathcal{N}$ independently determines its computing power allocation p_n . In the next section, we formulate the joint decision-making of the ESP and the MDs as a NBP.

³Tasks submitted to an application or edge-computing runtime can be managed through a shared queue and scheduled across multiple cores. The ESP can therefore treat the MD's aggregate computing capacity as one logical service queue. However, the proposed mechanism does not rely on the specific M/M/1 processing delay formulation and remains applicable under other processing delay models.

4. Nash Bargaining Problem Formulation and Problem Decomposition

The MDs are self-interested agents who participate in the system only if doing so increases their utilities. To incentivize cooperation between the ESP and the MDs, we model their joint decision-making as an NBP and define its NBS. Then, we decompose the NBP into an SWM problem and an ORD problem.

4.1. Nash Bargaining Problem Formulation

The ESP and the MDs cooperate to determine a feasible agreement $(\lambda, \mathbf{p}, \mathbf{r})$, including the task dispatch rates, computing power allocation, and reward distribution. If the ESP and the MDs fail to reach an agreement, they obtain their disagreement utilities. The ESP's disagreement utility is $\phi_0 = 0$. The disagreement utility of MD $n \in \mathcal{N}$ is $\phi_n = b_n(0)$, which is the utility obtained when it does not contribute any computing power to the ESP. Therefore, the ESP and the MDs are willing to cooperate only if their utilities are no lower than their disagreement utilities.

The NBS selects the feasible agreement that maximizes the weighted Nash product of the participants' excess utilities over their disagreement utilities. Its formal definition is as follows.

Definition 1. The strategy profile $(\lambda^*, \mathbf{p}^*, \mathbf{r}^*)$ is an NBS if it solves the following problem.

$$\max_{\lambda, \mathbf{p}, \mathbf{r}} (U_0(\lambda, \mathbf{p}, \mathbf{r}) - \phi_0)^{\omega_0} \prod_{n \in \mathcal{N}} (U_n(p_n, r_n) - \phi_n)^{\omega_n}, \quad (10a)$$

$$\text{s.t. } \sum_{n \in \mathcal{N}} \lambda_n = \lambda_0, \quad (10b)$$

$$\lambda_n \leq p_n / sl - \varepsilon_1, \forall n \in \mathcal{N}, \quad (10c)$$

$$p_n \leq F_n, \forall n \in \mathcal{N}, \quad (10d)$$

$$D_n(\lambda_n, p_n) \leq D_0 - \varepsilon_2, \forall n \in \mathcal{N}, \quad (10e)$$

$$r_n \geq 0, \forall n \in \mathcal{N}. \quad (10f)$$

The terms ω_0 and ω_n denote the bargaining power of the ESP and the MD $n \in \mathcal{N}$, respectively.

Constraint (10b) ensures that the total task dispatch rate equals the task arrival rate to the ESP. Constraints (10c) ensure that at each MD the task processing rate is higher than the task dispatch rate. We introduce a small positive constant ε_1 to relax the strict inequalities. Constraints (10d) ensure that each MD's allocated computing power does not exceed its computing power capacity. Constraints (10e) ensure the maximum delay does not exceed the ESCs' tolerance. We also introduce a small positive constant ε_2 to relax the strict inequalities. Finally, constraints (10f) ensure that each MD's reward is non-negative.

The bargaining power parameters ω_0 and ω_n are exogenous weights that characterize the relative negotiation priority of the ESP and MD n in sharing the cooperation surplus. Only the relative values of these weights matter in determining the NBS. In practice, bargaining power can

be specified by service agreements or platform rules before the solving bargaining problem. For example, the ESP may have a larger bargaining weight because it organizes the service and guarantees QoS, while the platform may assign an MD's bargaining weight based on its service tier or historical reliability. When such information is unavailable, equal bargaining weights can be used as a neutral default. We provide empirical sensitivity analysis of the bargaining power in Section 7.5.2.

The NBS satisfies the following desirable properties [37, 5]:

Individual Rationality. Both the ESP and the MDs obtain utilities no less than their disagreement utilities, i.e., what they would obtain without cooperation. This property ensures that neither the ESP nor the MDs are worse off by joining the cooperative computing power provision framework, which is crucial for self-interested MDs that may otherwise refuse to contribute their computing power.

Pareto Optimality. It is impossible to improve the utility of one participant without reducing the utility of at least one other participant. In our setting, this means that the allocation of task dispatch rates, computing power, and rewards achieves the highest possible system efficiency under the given constraints.

Proportional Fairness. Proportional fairness balances the relative excess utilities of the ESP and the MDs. Our framework takes into account each participant's bargaining power when determining how the cooperation surplus is shared. Such a balanced allocation prevents any participant from gaining disproportionately and helps sustain cooperation between the ESP and the MDs.

We equivalently transform the problem in (10) into an NBP by substituting the product of terms with the sum of the corresponding logarithms. We define $L_n(p_n) \triangleq E_n(p_n) + b_n(0) - b_n(p_n)$, $\forall n \in \mathcal{N}$, which is a convex function of p_n . We formulate the NBP as follows:

$$\begin{aligned} \text{(NBP)} \quad & \max_{\lambda, \mathbf{p}, \mathbf{r}} \omega_0 \log(Q(D_{\max}) - \sum_{n \in \mathcal{N}} r_n) \\ & + \sum_{n \in \mathcal{N}} \omega_n \log(r_n - L_n(p_n)), \end{aligned} \quad (11a)$$

$$\text{s.t. (10b) - (10f).}$$

The NBP is a convex problem. The feasible region defined by (10b)-(10f) is convex, since all constraints are either affine or convex in (λ_n, p_n, r_n) . The logarithmic objective function is concave due to the concavity of the ESP's and the MDs' utilities. Hence, the ESP can solve the NBP centrally using standard convex optimization techniques such as the interior-point methods or the primal-dual methods. However, this would incur significant computation overheads for the ESP, especially when the number of MDs is very large. Furthermore, the ESP needs complete information from all MDs, including private parameters such as sensitivity coefficient k_n , energy consumption coefficient c_n , and computing capacity F_n . Disclosing such information may reveal critical aspects of the MDs and potentially facilitate adversarial

attacks. Thus, considering the MDs' need for privacy protection, it is vital to design a distributed algorithm that requires limited private-parameter exchange.

Therefore, we seek to solve the NBP using distributed algorithms that require limited private-parameter exchange. To this end, we decompose the NBP into two subproblems, enabling the design of distributed algorithms.

4.2. Problem Decomposition

We observe an efficiency–fairness separation structure of the NBP. The task dispatch rates and computing-power allocations determine the total cooperation surplus created by the ESP–MD collaboration, while the rewards determine how this surplus is shared among the ESP and the MDs. Based on this observation, we decompose the NBP into a SWM problem and an ORD problem.

We define social welfare as the sum of the ESP and the MDs' excess utilities, i.e., $U_0 - \phi_0 + \sum_{n \in \mathcal{N}} U_n - \phi_n$. ESP's payment balances the MDs' total rewards in the social welfare. The reward decision variables $\mathbf{r}$ are thus eliminated. Hence, the SWM problem involves only task dispatch rates λ and computing power allocation $\mathbf{p}$:

$$\begin{aligned} \text{(SWM)} \quad & \max_{\lambda, \mathbf{p}} Q(D_{\max}) - \sum_{n \in \mathcal{N}} L_n(p_n), \quad (12a) \\ \text{s.t.} \quad & (10b) - (10e) \end{aligned}$$

Essentially, the SWM problem aims to maximize the utility surplus created from the ESP and the MDs' cooperation. Let S^* denote the optimal social welfare of the SWM problem. The NBP problem is well-defined only when S^* is positive. We make the following key assumption.

Assumption 1 (Positive Cooperative Surplus). *The optimal social welfare of the SWM problem satisfies $S^* > 0$.*

Assumption 1 implies that we focus on scenarios where cooperation between the ESP and the MDs create positive social welfare. This is both necessary and sufficient for the existence of an individually rational reward distribution for both the ESP and the MDs. We further provide feasibility conditions for Assumption 1 in Appendix D. In practice, the assumption holds when the delay-sensitive service generates sufficiently high revenue and the recruited MDs can provide idle computing power at sufficiently low energy and opportunity costs. It may fail in poor conditions, for example, the service revenue is low, the delay requirement is overly stringent, wireless links are poor, or the MDs' costs are high.

We now present the following result to establish the connection between the NBP and the SWM problem.

Proposition 1. *Under Assumption 1, the optimal task dispatch rates and computing power allocation for the NBP are the optimal solution to the SWM problem, i.e., achieving the optimal social welfare S^* .*

Proof. The proof is given in Appendix A. $\square$

Proposition 1 shows that the cooperation between the ESP and the MDs under the NBS maximizes the social

welfare of the system. This result motivates us to derive the optimal task dispatch rates and computing power allocation by solving the SWM problem.

Next, it remains to obtain the optimal reward distribution. Given the optimal task dispatch rates λ^* and the optimal computing power allocation $\mathbf{p}^*$, we define the ORD problem as follows. It finds the optimal rewards that maximize the Nash product of excess utilities:

$$\begin{aligned} \text{(ORD)} \quad & \max_{\mathbf{r}} \omega_0 \log(U_0(\lambda^*, \mathbf{p}^*, \mathbf{r}) - \phi_0) \\ & + \sum_{n \in \mathcal{N}} \omega_n \log(U_n(p_n^*, r_n) - \phi_n) \quad (13) \\ \text{s.t.} \quad & r_n \geq 0, \forall n \in \mathcal{N}. \end{aligned}$$

We formally show the equivalence between the NBP and two subproblems (i.e., the SWM problem and the ORD problem).

Proposition 2. *Under Assumption 1, the optimal solution $\lambda^*, \mathbf{p}^*$ to the SWM problem and the optimal solution $\mathbf{r}^*$ to the ORD problem are the optimal solution to the NBP.*

Proof. The proof is given in Appendix B. $\square$

Proposition 2 reveals a structural property of the NBP in our cooperative computing-power provision framework. The SWM problem determines the efficiency side of cooperation, creating the maximum cooperation surplus. Meanwhile, the ORD problem determines the fairness side of cooperation, fairly distributing the created surplus as rewards to MD. Together, they exactly recover the solution to the original NBP, as established in Proposition 2.

Therefore, we can solve the NBP in two steps. In the first step, the ESP and the MDs cooperate to maximize social welfare and determine the optimal task dispatch rates and computing power allocation by solving the SWM problem. In the second step, the ESP and the MDs bargain to decide the optimal distribution of rewards by solving the ORD problem.

In the following sections, we design distributed algorithms to solve the SWM problem and the ORD problem, respectively. In Section 5, we first propose an ADMM-based distributed algorithm to solve the SWM problem. Then in Section 6, we propose a distributed negotiation algorithm to solve the ORD problem.

5. ADMM-based Social Welfare Maximization

In this section, we solve the SWM problem and derive the optimal task dispatch rates and computing power allocation. A centralized solution incurs significant computation overheads for the ESP and requires the MDs' private information, which is involved in the objective function and the constraints. To address these limitations, we propose an ADMM-based distributed algorithm to decompose the SWM problem into $N + 1$ subproblems, one for each MD

and the ESP. This enables the system to maximize social welfare in a distributed manner and requires limited private-parameter exchange.

5.1. Problem Transformation

Recall that the social welfare relies on the maximum delay D_{max} across all MDs. To remove the non-smooth max operator in D_{max} , with a slight abuse of notation, we introduce an auxiliary variable $\hat{D}_{max}$, which denotes the system-wide delay upper bound. Constraints (10e) can be rewritten as:

$$D_{max} \leq D_0 - \varepsilon_2, \quad (14)$$

$$D_n(\lambda_n, p_n) \leq D_{max}, \quad \forall n \in \mathcal{N}. \quad (15)$$

The SWM problem is transformed into the following problem.

$$\begin{aligned} \text{(T-SWM)} \quad & \max_{\lambda, p, D_{max}} Q(D_{max}) - \sum_{n \in \mathcal{N}} L_n(p_n) \\ \text{s.t.} \quad & (10b), (10c), (10d), (14), (15). \end{aligned} \quad (16)$$

The T-SWM problem is a convex optimization problem that admits a centralized solution. But this incurs high computational overheads and requires the MDs' private information. To address these issues, we decompose the T-SWM problem into independent subproblems, enabling the ESP and the MDs to solve their own subproblems in a distributed manner.

5.2. ADMM-based Problem Decomposition

To solve the T-SWM problem with decision variables λ, p , and D_{max} , we let the ESP optimize the task dispatch rates λ and the delay upper bound D_{max} , and let each MD $n \in \mathcal{N}$ optimize its own computing power allocation p_n locally.

However, the T-SWM problem involves coupling constraints that contain the decision variables of the ESP and the MDs. Specifically, constraints (10c) couple λ_n and p_n and constraints (15) further couple λ_n, p_n and D_{max} . These couplings complicate the decomposition of the T-SWM problem.

To make the problem decomposable, we introduce auxiliary variables $\hat{\lambda}_n$ and $\hat{D}_{max,n}$ as local copies of variables λ_n and D_{max} , respectively. Each MD $n \in \mathcal{N}$ holds a pair of auxiliary variables $\hat{\lambda}_n$ and $\hat{D}_{max,n}$:

$$\hat{\lambda}_n = \lambda_n, \quad \forall n \in \mathcal{N}, \quad (17)$$

$$\hat{D}_{max,n} = D_{max}, \quad \forall n \in \mathcal{N}. \quad (18)$$

Based on these equations, the coupling constraints (10c) and (15) can be rewritten as

$$\hat{\lambda}_n \leq p_n / sl - \varepsilon_1, \quad \forall n \in \mathcal{N}, \quad (19)$$

$$D_n(\hat{\lambda}_n, p_n) \leq \hat{D}_{max,n}, \quad \forall n \in \mathcal{N}. \quad (20)$$

Let $\hat{\lambda} \triangleq [\hat{\lambda}_1, \dots, \hat{\lambda}_N]$ and $\hat{D}_{max} \triangleq [\hat{D}_{max,1}, \dots, \hat{D}_{max,N}]$. The T-SWM problem can be equivalently rewritten as:

$$\begin{aligned} \text{(R-SWM)} \quad & \max_{\lambda, p, D_{max}, \hat{\lambda}, \hat{D}_{max}} Q(D_{max}) - \sum_{n \in \mathcal{N}} L_n(p_n) \\ \text{s.t.} \quad & (10b), (10d), (14), (17), (18), (19), (20). \end{aligned} \quad (21)$$

s.t. (10b), (10d), (14), (17), (18), (19), (20).

Such a reformulation separates the coupling between the ESP's decision variables and those of the MDs, while preserving equivalence to the original T-SWM problem. This facilitates the decomposition of the problem, and allows the ESP and the MDs to solve their respective subproblems in a distributed manner.

Now, we decompose the R-SWM problem into independent subproblems with the ADMM method. The augmented Lagrangian function of the R-SWM problem is defined as

$$\begin{aligned} \mathcal{L}^\rho(\lambda, p, D_{max}, \hat{\lambda}, \hat{D}_{max}, \alpha, \beta) = & \sum_{n \in \mathcal{N}} L_n(p_n) - Q(D_{max}) \\ & + \sum_{n \in \mathcal{N}} \alpha_n(\hat{\lambda}_n - \lambda_n) + \frac{\rho}{2} \sum_{n \in \mathcal{N}} (\hat{\lambda}_n - \lambda_n)^2 \\ & + \sum_{n \in \mathcal{N}} \beta_n(\hat{D}_{max,n} - D_{max}) + \frac{\rho}{2} \sum_{n \in \mathcal{N}} (\hat{D}_{max,n} - D_{max})^2, \end{aligned} \quad (22)$$

where $\alpha = [\alpha_1, \dots, \alpha_N]$ and $\beta = [\beta_1, \dots, \beta_N]$ are the Lagrangian multipliers of constraints (17) and (18), respectively. Parameter ρ is a positive augmented Lagrangian parameter that controls the convergence of the ADMM.

The ESP and MDs jointly optimize the augmented Lagrangian function (22) by iteratively updating their own decision variables in an alternating manner. Specifically, in each iteration, the ESP first optimizes global variables λ and D_{max} . Then, each MD $n \in \mathcal{N}$ optimizes its local variables $p_n, \hat{\lambda}_n$ and $\hat{D}_{max,n}$ in parallel. Finally, the ESP updates the dual variables α, β . Next, we present the optimization procedures for the ESP's global variables in Section 5.3, the MDs' local variables in Section 5.4, and the dual variables in Section 5.5.

5.3. Optimization of ESP's Global Variables

At the beginning of each iteration t , the ESP minimizes the augmented Lagrangian function in (22) with respect to the task dispatch rates λ and the system-wide delay upper bound D_{max} . We denote the values of the MDs' auxiliary variables and the dual variables in iteration $t-1$ as $\hat{\lambda}^{[t-1]}, \hat{D}_{max}^{[t-1]}$ and $\alpha^{[t-1]}, \beta^{[t-1]}$, respectively. Given the values $\hat{\lambda}^{[t-1]}, \hat{D}_{max}^{[t-1]}, \alpha^{[t-1]}$, and $\beta^{[t-1]}$, the ESP's global subproblem is defined as

$$\begin{aligned} \min_{\lambda, D_{max}} \quad & \mathcal{L}_0^\rho(\lambda, D_{max}, \hat{\lambda}^{[t-1]}, \hat{D}_{max}^{[t-1]}, \alpha^{[t-1]}, \beta^{[t-1]}) \\ \text{s.t.} \quad & (10b), (14), \end{aligned} \quad (23)$$

where the objective function is given by

$$\begin{aligned} \mathcal{L}_0^\rho(\lambda, D_{max}, \hat{\lambda}^{[t-1]}, \hat{D}_{max}^{[t-1]}, \alpha^{[t-1]}, \beta^{[t-1]}) = & -Q(D_{max}) \\ & + \sum_{n \in \mathcal{N}} \alpha_n^{[t-1]}(\hat{\lambda}_n^{[t-1]} - \lambda_n) + \frac{\rho}{2} \sum_{n \in \mathcal{N}} (\hat{\lambda}_n^{[t-1]} - \lambda_n)^2 \\ & + \sum_{n \in \mathcal{N}} \beta_n^{[t-1]}(\hat{D}_{max,n}^{[t-1]} - D_{max}) + \frac{\rho}{2} \sum_{n \in \mathcal{N}} (\hat{D}_{max,n}^{[t-1]} - D_{max})^2. \end{aligned} \quad (24)$$

The objective function in (24) actually corresponds to the terms of the augmented Lagrangian function in (22) that depends on λ , $D_{\max}$, $\hat{\lambda}^{[t-1]}$, $\hat{D}_{\max}^{[t-1]}$, $\alpha^{[t-1]}$, and $\beta^{[t-1]}$.

The ESP's global subproblem in (23) is a convex problem and can be solved using standard convex optimization methods, such as the interior point method or the primal-dual method.

After solving the problem in (23), the ESP obtains the values of its global variables $\lambda^{[t]}$, $D_{\max}^{[t]}$ in the t -th iteration. It then broadcasts $\lambda_n^{[t]}$, $D_{\max}^{[t]}$, $\alpha_n^{[t-1]}$, and $\beta_n^{[t-1]}$ to each MD n . Subsequently, the MDs use these values to update their local variables in parallel.

5.4. Optimization of MD's Local Variables

In each iteration t , the MDs collectively minimize the augmented Lagrangian function in (22) with respect to the computing power allocation p and the auxiliary variables $\hat{\lambda}$, $\hat{D}_{\max}$. These variables are separate across the MDs, and each MD $n \in \mathcal{N}$ can minimize its p_n , $\hat{\lambda}_n$ and $\hat{D}_{\max,n}$ in parallel.

Upon receiving the values $\lambda_n^{[t]}$, $D_{\max}^{[t]}$, $\alpha_n^{[t-1]}$, and $\beta_n^{[t-1]}$ from the ESP, each MD n solves its local subproblem as follows.

$$\min_{p_n, \hat{\lambda}_n, \hat{D}_{\max,n}} \mathcal{L}_n^\rho(p_n, \hat{\lambda}_n, \hat{D}_{\max,n}, \lambda_n^{[t]}, D_{\max}^{[t]}, \alpha_n^{[t-1]}, \beta_n^{[t-1]}) \quad (25a)$$

$$s.t. \ p_n \leq F_n, \quad (25b)$$

$$\hat{\lambda}_n \leq p_n / sl - \epsilon_1, \quad (25c)$$

$$D_n(\hat{\lambda}_n, p_n) \leq \hat{D}_{\max,n}. \quad (25d)$$

The objective function is given by

$$\begin{aligned} & \mathcal{L}_n^\rho(p_n, \hat{\lambda}_n, \hat{D}_{\max,n}, \lambda_n^{[t]}, D_{\max}^{[t]}, \alpha_n^{[t-1]}, \beta_n^{[t-1]}) \\ &= L_n(p_n) + \alpha_n^{[t-1]}(\hat{\lambda}_n - \lambda_n^{[t]}) + \frac{\rho}{2}(\hat{\lambda}_n - \lambda_n^{[t]})^2 \\ &+ \beta_n^{[t-1]}(\hat{D}_{\max,n} - D_{\max}^{[t]}) + \frac{\rho}{2}(\hat{D}_{\max,n} - D_{\max}^{[t]})^2. \end{aligned} \quad (26)$$

The objective function in (26) actually corresponds to the terms of the augmented Lagrangian function in (22) that depends on p_n , $\hat{\lambda}_n$, $\hat{D}_{\max,n}$, $\lambda_n^{[t]}$, $D_{\max}^{[t]}$, $\alpha_n^{[t-1]}$, and $\beta_n^{[t-1]}$.

The local subproblem in (25) for each MD n is convex. As a result, each MD n can obtain the solution $(p_n^{[t]}, \hat{\lambda}_n^{[t]}, \hat{D}_{\max,n}^{[t]})$ using standard methods in convex optimization. Upon solving the subproblem, the MD n sends the solution back to the ESP, who then proceeds to update the dual variables.

5.5. The Update of Dual Variables

After obtaining the global variables and the local variables in iteration t , the ESP updates the dual variables to maximize the augmented Lagrangian function in (22) through gradient ascent as follows.

$$\alpha_n^{[t]} = \alpha_n^{[t-1]} + \rho(\hat{\lambda}_n^{[t]} - \lambda_n^{[t]}), \quad \forall n \in \mathcal{N}, \quad (27)$$

$$\beta_n^{[t]} = \beta_n^{[t-1]} + \rho(\hat{D}_{\max,n}^{[t]} - D_{\max}^{[t]}), \quad \forall n \in \mathcal{N}. \quad (28)$$

To summarize, the proposed ADMM-based approach solves the R-SWM problem by alternately updating the ESP's global variables, the MDs' local variables, and the dual variables, thereby minimaxing the augmented Lagrangian function. Specifically, the ESP first optimizes the task dispatch rates and the system-wide delay upper bound. Next, each MD optimizes its computing power allocation and the auxiliary variables in parallel. Finally, the ESP updates the dual variables. This approach frees the ESP from heavy computation overheads and requires limited private-parameter exchange. The procedure of the distributed ADMM-based social welfare maximization algorithm is summarized in Algorithm 1.

We highlight that Algorithm 1 requires limited private-parameter exchange. Each MD only needs to return its local variables to the ESP. Moreover, each MD solves its local subproblem independently, using only its local information, and the global and dual variables broadcast by the ESP. The algorithm does not require information exchange among the MDs.

5.6. Performance Analysis of Algorithm 1

In this subsection, we analyze the convergence behavior and overall complexity of our distributed ADMM-based social welfare maximization algorithm in Algorithm 1.

5.6.1. Convergence Result

We establish that Algorithm 1 converges to the optimal objective value of the SWM problem as the number of iterations tends to infinity.

Recall that $S^* > 0$ denotes the optimal object value of the SWM problem. Let $S^{[t]}$ denote the objective value of the R-SWM problem in iteration t , i.e., $S^{[t]} = Q(D_{\max}^{[t]}) - \sum_{n \in \mathcal{N}} L_n(p_n^{[t]})$. The following theorem establishes the convergence of Algorithm 1 in terms of objective value:

Theorem 1. *Under Assumption 1, the sequence of objective values $\{S^{[t]}\}$ produced by Algorithm 1 converges to the optimal value S^* of the SWM problem, i.e., $S^{[t]} \rightarrow S^*$ as $t \rightarrow \infty$.*

Proof. The proof is provided in Appendix C. $\square$

5.6.2. Complexity Analysis

We characterize the computational complexity of Algorithm 1 in terms of the per-iteration complexity and the iteration complexity, respectively.

Per-iteration Complexity. Each iteration of Algorithm 1 involves solving the ESP's global subproblem in (23), N MDs' local subproblems in (25), and updating the dual variables. Both subproblems in (23) and (25) are convex. Therefore, various solution methods can be applied. In the following analysis, we consider the classical interior-point method.

In the ESP's global subproblem in (23), there are totally $N + 1$ optimization variables and two linear constraints. The interior-point method requires $O(\sqrt{2} \log(1/\epsilon))$ iterations to achieve a precision of ϵ [38]. Each Newton step takes

Algorithm 1 Distributed ADMM-based Social Welfare Maximization**Input:** Termination criterion threshold $\epsilon > 0$.**Output:** $D_{\max}^*, \lambda^*, p^*$

- 1: Initialize decision variables $D_{\max}^{[0]}, \lambda^{[0]}, p^{[0]}$, auxiliary variables $\hat{\lambda}^{[0]}, \hat{D}_{\max}^{[0]}$, dual variables $\alpha^{[0]}, \beta^{[0]}$;
- 2: **for** $t = 1, 2, \dots$, **do**
- 3: The ESP computes $\lambda^{[t]}, D_{\max}^{[t]}$ by solving the global subproblem in (23);
- 4: The ESP broadcasts $\lambda_n^{[t]}, D_{\max}^{[t]}, \alpha_n^{[t-1]}$, and $\beta_n^{[t-1]}$ to each MD $n \in \mathcal{N}$;
- 5: Each MD n computes $p_n^{[t]}, \hat{\lambda}_n^{[t]}, \hat{D}_{\max,n}^{[t]}$ by solving the local subproblem in (25), and sends them back to the ESP;
- 6: The ESP receives $p^{[t]}, \hat{\lambda}^{[t]}, \hat{D}_{\max}^{[t]}$. Then it computes $\alpha^{[t]}, \beta^{[t]}$ via (27) and (28);
- 7: The ESP checks the termination criterion:
- 8: **if** $|D_{\max}^{[t]} - D_{\max}^{[t-1]}| < \epsilon$, $|\lambda_n^{[t]} - \lambda_n^{[t-1]}| < \epsilon, \forall n \in \mathcal{N}$ and $|p_n^{[t]} - p_n^{[t-1]}| < \epsilon, \forall n \in \mathcal{N}$ **then**
- 9: break loop and go to step 12;
- 10: **end if**
- 11: **end for**
- 12: Output the optimal delay upper bound $D_{\max}^* = D_{\max}^{[t]}$, the optimal task dispatch rates $\lambda^* = \lambda^{[t]}$ and the optimal computing power allocation $p^* = p^{[t]}$;

$O((N+1)^3)$ arithmetic operations. Therefore, the complexity of solving the ESP's global subproblem is $O(N^3 \log(1/\epsilon))$ under a precision of ϵ .

Each MD's local subproblem in (25) involves three optimization variables and three convex constraints, so its interior-point solution likewise takes $O(\log(1/\epsilon))$ iterations, with each Newton step costing $O(1)$ operations. Since all MDs solve their subproblems in parallel, the complexity of solving all MDs' local subproblems remains $O(\log(1/\epsilon))$.

The ESP updates dual variables based on equations (27) and (28). The complexity is thus $O(N)$. Therefore, overall per-iteration complexity of Algorithm 1 is $O(N^3 \log(1/\epsilon) + \log(1/\epsilon) + N) = O(N^3 \log(1/\epsilon))$.

Iteration Complexity. Algorithm 1 is a classical two-block ADMM: the first block comprises the ESP's global variables $(\lambda, D_{\max})$, whereas the second block includes all MDs' local variables $(p, \hat{\lambda}, \hat{D}_{\max})$ updated in parallel. Let $\mathbf{w}^{[t]}$ denote the stacked primal-dual vector in the t -th iteration. The non-ergodic convergence result in [19] implies

$$\|\mathbf{w}^{[t+1]} - \mathbf{w}^{[t]}\|_H^2 \leq \frac{C}{t+1}, \quad t = 0, 1, \dots, \quad (29)$$

for some constant matrix $H \succ 0$ and scalar $C > 0$. To enforce the step gap $\|\mathbf{w}^{[t+1]} - \mathbf{w}^{[t]}\|_H^2 \leq \epsilon$, one therefore needs

$$t \geq \frac{C}{\epsilon} - 1, \quad (30)$$

so Algorithm 1 achieves an iteration complexity of $\mathcal{O}(1/\epsilon)$.

Communication Overhead. In each ADMM iteration, the ESP sends four scalar quantities to each MD n : $\lambda_n^{[t]}$, $D_{\max}^{[t]}$, $\alpha_n^{[t-1]}$, and $\beta_n^{[t-1]}$. Each MD then returns three scalar quantities to the ESP: $p_n^{[t]}$, $\hat{\lambda}_n^{[t]}$, and $\hat{D}_{\max,n}^{[t]}$. Thus, each iteration involves $2N$ one-hop messages and $7N$ scalar values in total. The communication overhead is therefore $O(N)$ per iteration and scales linearly with the number of MDs. Combining this with the iteration complexity, the total scalar communication overhead until convergence is $O(N/\epsilon)$ under the stopping precision ϵ .

6. Distributed Negotiation for Optimal Reward Distribution

In this section, we focus on the ORD problem in (13) and solve for the optimal reward distribution between the ESP and the MDs. Given the optimal computing power allocation p^* and the maximum delay $D_{\max}^*$ obtained from solving the SWM problem, we denote $Q^* \triangleq Q(D_{\max}^*)$ and $L_n^* \triangleq L_n(p_n^*)$, $\forall n \in \mathcal{N}$. We introduce an auxiliary variable r_0 for the ESP to denote the total reward paid to the MDs. The ORD problem can be equivalently restated in the following form.

$$\max_{r_0, \mathbf{r}} \omega_0 \log(Q^* - r_0) + \sum_{n \in \mathcal{N}} \omega_n \log(r_n - L_n^*) \quad (31a)$$

$$s.t. \ r_0 \geq 0, \quad (31b)$$

$$r_n \geq 0, \forall n \in \mathcal{N}, \quad (31c)$$

$$\sum_{n \in \mathcal{N}} r_n = r_0. \quad (31d)$$

The above problem in (31) is convex. Constraint (31d) ensures that the total reward paid by the ESP equals the sum of the individual rewards received by all MDs. We introduce $\gamma \geq 0$ as the Lagrangian multiplier associated with constraint (31d). The Lagrangian function of problem (31) is:

$$\begin{aligned} \mathcal{L}_R(r_0, \mathbf{r}, \gamma) = & -\omega_0 \log(Q^* - r_0) - \sum_{n \in \mathcal{N}} \omega_n \log(r_n - L_n^*) \\ & + \gamma \left(\sum_{n \in \mathcal{N}} r_n - r_0 \right). \end{aligned} \quad (32)$$

We do not explicitly include the non-negativity constraints (31b) and (31c) in the Lagrangian function in (32), as we will show later that r_n and r_0 are indeed non-negative.

Applying the first-order optimality conditions to the Lagrangian function $\mathcal{L}_R(r_0, \mathbf{r}, \gamma)$ with respect to r_0 and $\mathbf{r}$ yields the following closed-form expressions:

$$r_0^*(\gamma) = Q^* - \frac{\omega_0}{\gamma}, \quad (33)$$

$$r_n^*(\gamma) = L_n^* + \frac{\omega_n}{\gamma}, \forall n \in \mathcal{N}. \quad (34)$$

Solving the associated dual optimization problem $\max_{\gamma} \mathcal{L}_R(r_0^*(\gamma), \mathbf{r}^*(\gamma), \gamma)$ using standard convex optimization techniques (e.g., dual ascent or subgradient descent) requires carefully tuned step sizes and many iterations, making the

Algorithm 2 Distributed Negotiation for Optimal Reward Distribution

Input: Initial upper bound M of the Lagrangian multiplier γ , tolerance threshold ϵ .

Output: $\mathbf{r}^*$

- 1: Initialize a tolerance threshold ϵ . Initialize the upper bound for the Lagrangian multiplier γ as $\bar{\gamma}^{[0]} = M$, and its lower bound $\underline{\gamma}^{[0]} = \frac{\omega_0}{S^*}$;
- 2: **for** $t = 1, 2, \dots$ **do**
- 3: The ESP sets $\gamma^{[t]} = \frac{1}{2}(\bar{\gamma}^{[t-1]} + \underline{\gamma}^{[t-1]})$ and calculate $r_0^{[t]} = Q^* - \frac{\omega_0}{\gamma^{[t]}}$.
- 4: The ESP broadcasts the Lagrangian multiplier $\gamma^{[t]}$ to all MDs.
- 5: Each MD n receives $\gamma^{[t]}$ and evaluates its reward $r_n^{[t]} = L_n^* + \frac{\omega_n}{\gamma^{[t]}}$.
- 6: Each MD n reports $r_n^{[t]}$ to the ESP.
- 7: The ESP receives all MDs' reports and compares $r_0^{[t]}$ and $\sum_{n \in \mathcal{N}} r_n^{[t]}$. If $r_0^{[t]} > \sum_{n \in \mathcal{N}} r_n^{[t]} + \epsilon$, then set $\bar{\gamma}^{[t]} = \gamma^{[t]}$, $\underline{\gamma}^{[t]} = \underline{\gamma}^{[t-1]}$. If $r_0^{[t]} < \sum_{n \in \mathcal{N}} r_n^{[t]} - \epsilon$, then set $\bar{\gamma}^{[t]} = \bar{\gamma}^{[t-1]}$, $\underline{\gamma}^{[t]} = \gamma^{[t]}$. Otherwise, output $\mathbf{r}^* = \mathbf{r}^{[t]}$ as the optimal reward distribution.
- 8: **end for**

dual maximization computationally heavy. Instead, we observe that any optimal solution to the problem in (31) must satisfy the equality constraint (31d). By exploiting this property, we reduce the search for γ to a one-dimensional root-finding problem, which can be solved efficiently by a simple bisection procedure.

Recall that we denote S^* as the optimal social welfare in Section 4 and we have $S^* = Q^* - \sum_{n \in \mathcal{N}} L_n^*$. Define $W \triangleq \omega_0 + \sum_{n \in \mathcal{N}} \omega_n$, and $\Delta(\gamma) \triangleq r_0^*(\gamma) - \sum_{n \in \mathcal{N}} r_n^*(\gamma)$. Combining equations (31d), (33), and (34), we have:

$$\Delta(\gamma) = S^* - \frac{W}{\gamma} = 0. \quad (35)$$

That is, with the optimal primal variables $\mathbf{r}^*$, the optimal dual variable γ^* should satisfy $\gamma^* = W/S^*$. Although this gives a closed-form expression of γ^* , computing it centrally requires the ESP to collect all MD-specific quantities involved in W and S^* , including the bargaining power ω_n and the utility-loss terms L_n^* . To avoid centralized collection of such quantities, we design a distributed negotiation procedure to identify γ^* . We observe that the function $\Delta(\gamma)$ is a monotone function with respect to γ . This property enables us to adopt a bisection search to identify the unique γ^* that satisfies $\Delta(\gamma) = 0$.

To this end, we propose a bisection-based distributed negotiation algorithm in Algorithm 2. Specifically, in each round, the ESP proposes a value for the Lagrangian multiplier γ , and the MDs respond with their corresponding rewards. The ESP then adjusts the search range for γ until the computed values of $r_0^*(\gamma)$ and $\sum_{n \in \mathcal{N}} r_n^*(\gamma)$ are equal. This

approach enables the ESP and the MDs to collaboratively determine the optimal value of γ^* .

The initial upper bound of γ only requires a value M such that $M > \gamma^*$, where γ^* is the optimal Lagrangian multiplier satisfying $\Delta(\gamma^*) = 0$. Therefore, any sufficiently large M satisfying $r_0^*(M) > \sum_{n \in \mathcal{N}} r_n^*(M)$ can be served as a valid upper bound for the bisection procedure. In our implementation, we use a conservative fixed value $M = 10^5$. Before starting the bisection search, the ESP can verify the validity of this upper bound by broadcasting M to the MDs. Each MD computes $r_n^*(M) = L_n^* + \omega_n/M$ locally and reports it to the ESP. The ESP then checks whether $r_0^*(M) = Q^* - \omega_0/M$ is larger than $\sum_{n \in \mathcal{N}} r_n^*(M)$. If this condition holds, M is a valid upper bound. If the condition does not hold, the ESP can simply enlarge M , e.g., by doubling it, until the condition is satisfied. In all simulation scenarios considered in Section 7, $M = 10^5$ satisfies this condition.

Moreover, we show that $\mathbf{r}$ and r_0 are indeed positive. To see this, we notice that γ is positive. According to (34), we can see that $r_n^*(\gamma), \forall n \in \mathcal{N}$ are positive. And equation (31d) gives positive r_0 .

We highlight that Algorithm 2 requires limited private-parameter exchange, and is easy to implement in a distributed manner. To perform the updates in each iteration, the ESP uses its bargaining power ω_0 and resulting service revenue Q^* , while each MD n uses its bargaining power ω_n and resulting utility loss L_n^* . Both Q^* and L_n^* can be computed locally by the respective agents after executing Algorithm 1.

We further highlight that Algorithm 2 is highly efficient. Each iteration of the algorithm incurs only $O(1)$ computational complexity per agent, and $O(N)$ total communication overhead, making it highly scalable and practical for real-world deployments.

We characterize the convergence of Algorithm 2 as follows.

Theorem 2. Under Assumption 1, Algorithm 2 converges to the optimal solution to the ORD problem within $O\left(\log_2 \frac{M - \frac{\omega_0}{S^*}}{\epsilon}\right)$ rounds of iterations.

Proof. Algorithm 2 performs a bisection search over the one-dimensional variable γ . The upper bound $\bar{\gamma}$ is initially set as M . The lower bound $\underline{\gamma} = \omega_0/S^*$, which is strictly smaller than $\gamma^* = W/S^*$. Knowing the upper bound and the lower bound for γ , the bisection search in Algorithm 2 is guaranteed to converge to the optimum of the ORD problem within $O\left(\log_2 \frac{M - \frac{\omega_0}{S^*}}{\epsilon}\right)$ rounds of iterations. $\square$

7. Numerical Results

In this section, we first evaluate the proposed method under real-world task arrival and MD-mobility. We demonstrate its effectiveness in maximizing social welfare while maintaining low delay. We further highlight the advantages of cooperation between the ESP and the MDs, and perform sensitivity analysis.

7.1. Experimental Setup

Similar to existing studies [43, 15, 20], we set the parameters in our experiments as follows. The number of MDs and the task-arrival rate are specified separately for each experiment. Task parameters are as follows. The expected data size of each task s is 100 KB, and the expected computation intensity $l = 100$ cycle/bit [15]. The delay tolerance threshold D_0 is 1 second.

For each MD, the energy consumption coefficient c_n is uniformly sampled from 0.8×10^{-18} to 1.2×10^{-18} [36]. The sensitivity k_n follows a uniform distribution in (0.1, 0.9). The local computing capacity F_n is uniformly distributed in [1, 5] GHz.

For the ESP, we adopt the following form of the ESP's revenue function [20, 43]:

$$Q(D_{\max}) = \bar{Q} - \frac{o}{D_0 - D_{\max}}. \quad (36)$$

Here, $\bar{Q}$ denotes the ideal service revenue obtained from completing the ESCs' tasks, and the term $o/(D_0 - D_{\max})$ captures the revenue loss caused by the ESCs' dissatisfaction with the worst-case delay. The parameter $o > 0$ measures the ESCs' sensitivity to delay degradation. A larger o means that the ESP suffers a larger revenue loss for the same increase in $D_{\max}$. We set $\bar{Q} = \lambda_0 \theta$, where $\theta = 1$ is the normalized ideal revenue per completed task. We set the delay-sensitivity parameter as $o = 10$. Moreover, we set the ESP's bargaining power $\omega_0 = 2$, and the MDs' bargaining power $\omega_n = 1$. We set the initial upper bound M used in Algorithm 2 as 10^5 .

Communication parameters are as follows [15]. The wireless bandwidth of the ESP B is 10 MHz. The transmit power of ESP ξ is 10 Watts. The background noise power σ_n is uniformly set as -174 dBm/Hz. The channel power gain g_n is $g_n = h_n^{fade} h_n^{loss} (d_0/d_n)^\varpi$, where h_n^{fade} is the small-scale fading channel power gain that follows an exponential distribution with unit mean, $h_n^{loss} = -40$ dB is the path-loss constant, $\varpi = 4$ is the path-loss exponent, $d_0 = 1$ meter is the reference distance, and d_n is the distance from MD n to the nearest base station of the ESP. We assume that d_n follows a uniform distribution between [100, 150] meters.

We compare our proposed methods with the following baseline methods:

- **Non-cooperation** [62, 33]: This baseline models the interaction between the ESP and the MDs as a Stackelberg game. The ESP acts as a leader that announces the task dispatch rates λ and a unit price π for computing power. Then each MD, as a follower, receives a reward πp_n proportional to its contributed computing power and decides its optimal p_n to maximize its own utility.
- **Contract-based method** [25, 28]: Following a contract-theoretic approach, the ESP designs a set of contracts $\{(\lambda_n, p_n, r_n)\}_{n=1}^K$, where we set $K = 5$. These contracts satisfy individual rationality. Each MD selects the contract that maximizes its utility, and the selected

contracts determine the resulting task dispatch rates, computing-power allocations, and rewards.

- **Proportional dispatch**: The ESP dispatches the tasks *proportional* to each MD's computing capacities, i.e., $\lambda_n = \lambda_0 F_n / \sum_i F_i, \forall n \in \mathcal{N}$. Each MD randomly selects a feasible computing power that satisfies the delay constraint. The ESP distributes the resulting social welfare among MDs proportionally to their capacities as their rewards.
- **Uniform dispatch**: The ESP dispatches incoming tasks and total reward *uniformly* among all MDs, i.e., $\lambda_n = \lambda_0 / N$ for every $n \in \mathcal{N}$. Each MD randomly selects a feasible computing power that satisfies the delay constraint. The ESP evenly splits the resulting social welfare among all MDs as their rewards.

7.2. Evaluation under Real-World Task-Arrival Trace

In this subsection, we evaluate the proposed method under a real-world task-arrival trace generated from Google ClusterData 2019 [57], which contains workload traces from Google's Borg cluster management system. To cope with this dynamic setting, we divide the time horizon into scheduling epochs. In each epoch, the proposed method is executed using the current demand level.

7.2.1. Evaluation Setup

We consider a pool of $N = 20$ MDs. We then use the Google ClusterData to introduce a real-world temporal task-arrival pattern. Specifically, we select a continuous six-hour interval from the data, which contains 27,815 job-instance submission records. We retain only the first submission of each task instance and ignore task resubmissions, resulting in 9,211 unique task arrivals. We discard all other trace information in the data, such as scheduling, completion, and machine records.

We divide the selected 6-hour task-arrival trace into 5-minute epochs and yield 72 epochs. Let the task count in epoch k be denoted as A_k . Across the 72 epochs, the task count A_k ranges from 31 to 339 tasks per 5-minute epoch. We numerically scale up the task arrival rate in each epoch to the range between 50 and 150 tasks/s while preserving the temporal pattern. We calculate the task-arrival rate $\lambda_{0,k}$ for epoch k as

$$\lambda_{0,k} = 50 + 100 \text{clip}\left(\frac{A_k - Q_{0.05}}{Q_{0.95} - Q_{0.05}}, 0, 1\right), \quad (37)$$

where $Q_{0.05} = 40.85$ and $Q_{0.95} = 242.70$ are the 5th and 95th percentiles of the task counts of all 72 epochs, respectively. We illustrate the original task counts in the Google ClusterData and the resulting task-arrival rates across the 72 epochs in Fig. 2.

After determining the task-arrival rates for each epoch, we generate the arrival timestamp for each task within each epoch. We first extract the time gaps between consecutive

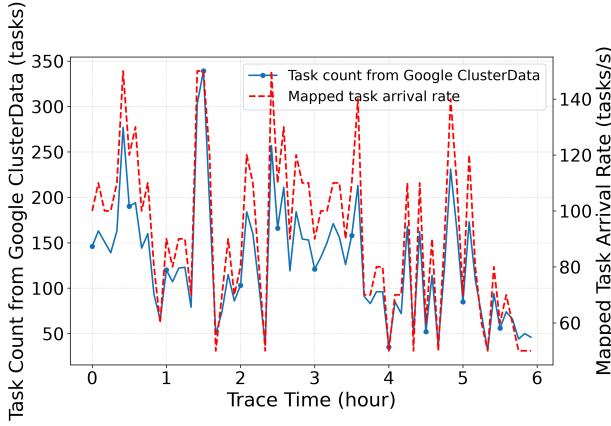


Figure 2: Task count from Google ClusterData and mapped task arrival rate for each 5-minute epoch in the selected 6-hour trace.

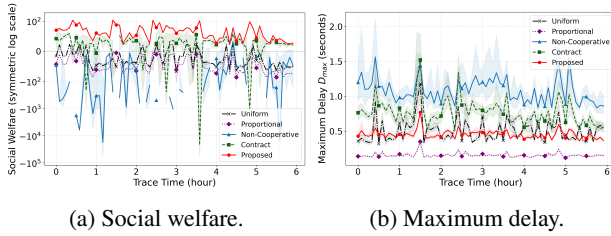


Figure 3: Social welfare and maximum delay in the selected 6-hour trace. Curves show average results and shaded regions show 95% intervals.

Table 1

Average maximum delay, social welfare, and percentage of epochs exceeding the delay limit in the selected 6-hour trace.

Method	Mean $D_{\max}$ (s)	Mean social welfare	Epochs with $D_{\max} \geq D_0$ (%)
Uniform	0.504	-28.392	1.94%
Proportional	0.157	-51.815	0.00%
Non-Cooperative	1.080	-959.723	55.83%
Contract	0.777	32.270	10.28%
Proposed	0.454	55.426	0.00%

task arrivals in the original 6-hour trace of Google ClusterData. We repeat these gaps to generate the required number of MEC tasks and then scale them proportionally to fit the 5-minute epoch. This allows the simulated tasks to follow the uneven arrival pattern observed in the original trace.

We replay the proposed method and the baseline methods under the setting in Section 7.1. All methods use the same list of generated tasks. For each task, we independently sample its computation workload from an exponential distribution with mean sI . The same sampled workload is used across all methods for a fair comparison. At the beginning of epoch k , each method computes its dispatch rates λ and computing-power allocation p from the task-arrival rate $\lambda_{0,k}$ for epoch k . Tasks are dispatched to each MD n with probabilities $\lambda_n/\lambda_{0,k}$. We repeat the experiments under 5 different random seeds.

7.2.2. Experimental Results

Figures 3a and 3b report social welfare and maximum delay over the six-hour trace, and Table 1 summarizes the performance of all methods. The proposed method achieves the highest mean social welfare with an average maximum delay of 0.454 s across the trace. Relative to the uniform, non-cooperation, and contract-based methods, it reduces the mean maximum delay by 9.9%, 58.0%, and 41.5%, respectively, while increasing the mean social welfare by 83.82, 1015.15, and 23.16. It also improves the mean social welfare by 107.24 over proportional dispatch. Although proportional dispatch attains a lower delay, the proposed method achieves substantially higher social welfare by balancing delay reduction against computing-power costs. Overall, the proposed method provides *the best welfare performance and a competitive delay under the time-varying workload in real-world edge environments*.

7.3. Evaluation under a Real-World MD-Mobility Trace

In this subsection, we evaluate the proposed method under dynamic MD mobility using a real-world connectivity trace from the Shanghai Telecom dataset [29, 17, 53], which contains mobile users' Internet access logs.

7.3.1. Evaluation Setup

We use the Internet access records at one selected base station in the Shanghai Telecom dataset to determine when each MD is available. Each anonymous user ID in the trace represents one MD in the experiment. Specifically, we use the records from June 1–15, 2014, and regard an MD as available in a five-minute epoch when one of its access intervals at the selected base station overlaps that epoch. The trace therefore determines the set $\mathcal{N}_t$ of MDs.

We select three six-hour windows in which MDs join or leave at low, moderate, and high frequencies. We first retain base stations observed on all 15 days with at least 50 distinct users. Among the ten stations with the most access records, we select the station with the largest total connected-user time, located at latitude 31.146311 and longitude 121.399951. We divide each day into four non-overlapping six-hour windows and retain windows with at least three available MDs in every epoch. From these candidates, we select the windows closest to the 25th, 50th, and 75th percentiles of the number of times that MDs join or leave. This procedure yields Window 1 (June 7, 00:00–06:00), Window 2 (June 1, 12:00–18:00), and Window 3 (June 13, 12:00–18:00), respectively. Window 1 includes 6 MDs over the six-hour period, with 3–4 available in each epoch and 17 MD join/leave events (i.e., MDs entering or leaving the base station coverage). Window 2 includes 13 MDs over the six-hour period, with 3–7 available in each epoch and 34 MD join/leave events. Window 3 includes 25 MDs over the six-hour period, with 3–9 available in each epoch and 58 MD join/leave events.

We evaluate the proposed method and the baselines using the same set of active MDs in each epoch of the trace.

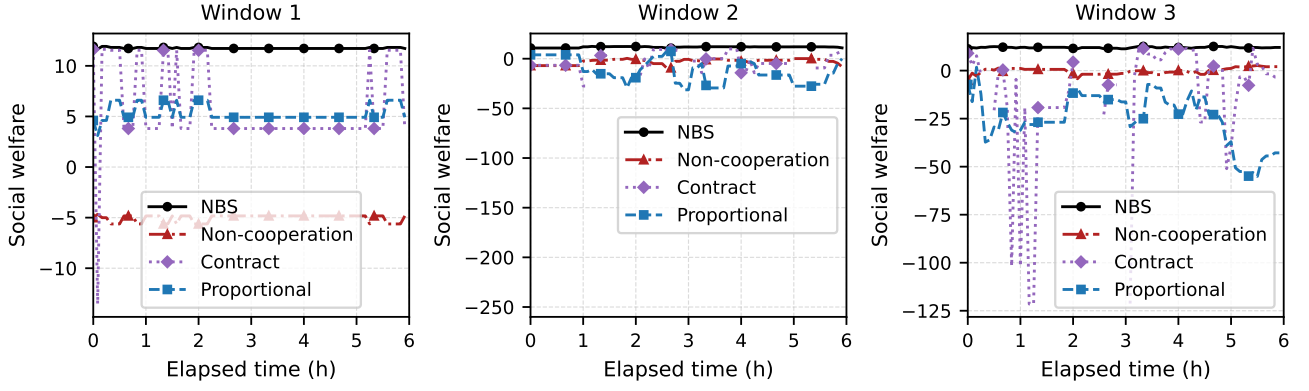


Figure 4: Social welfare in the three selected six-hour windows.

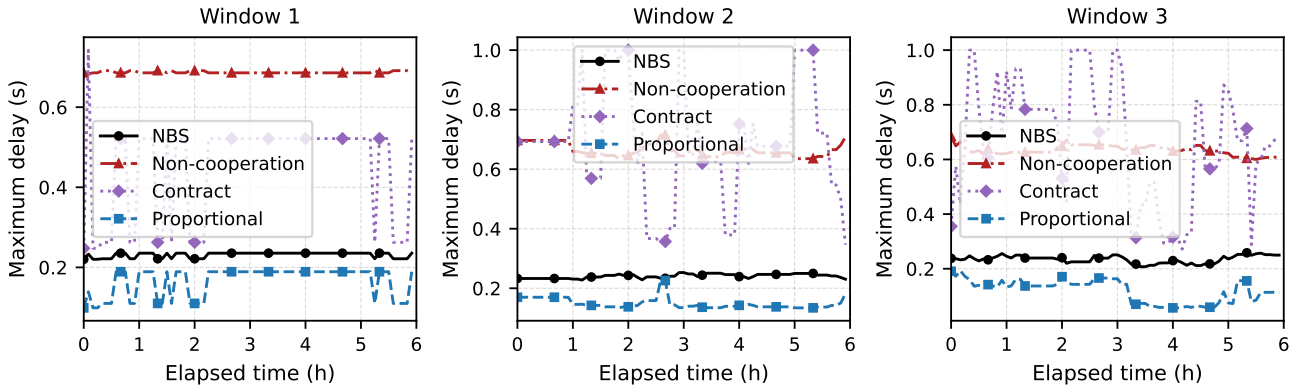


Figure 5: Maximum delay in the three selected six-hour windows.

Table 2

Mean social welfare, maximum delay, and computing power in the three selected six-hour windows.

	Window 1			Window 2			Window 3		
	S	$D_{\max}$ (s)	$\bar{p}$ (GHz)	S	$D_{\max}$ (s)	$\bar{p}$ (GHz)	S	$D_{\max}$ (s)	$\bar{p}$ (GHz)
Proposed	11.750	0.232	1.029	11.698	0.241	0.872	12.022	0.235	0.848
Non-Cooperative	-5.021	0.687	0.775	-2.806	0.664	0.637	-0.168	0.634	0.613
Contract	5.716	0.452	1.007	-6.984	0.621	0.762	-13.006	0.624	0.703
Proportional	5.275	0.166	1.593	-12.147	0.148	1.997	-24.942	0.120	2.502
Uniform	-504.169	0.947	1.792	-338.234	0.845	1.355	-193.110	0.523	2.251

Each window consists of 72 five-minute epochs, and we fix the task-arrival rate at $\lambda_0 = 30$ tasks/s. All other parameters for the ESP, tasks, and MDs follow Section 7.1.

7.3.2. Experimental Results

Table 2 and Figs. 4–5 report social welfare, maximum delay, and average computing power under the three selected six-hour traces. The uniform baseline results are reported in Table 2 but omitted from the figures, as their substantially larger negative welfare values would compress the other four curves. Compared with the contract baseline, the proposed method increases mean social welfare by 6.033, 18.598, and 25.045 and reduces mean maximum delay by 48.8%,

61.3%, and 62.4%. Relative to non-cooperation, it increases mean social welfare by 16.771, 14.504, and 12.190 and reduces mean maximum delay by 66.3%, 63.7%, and 63.0%, respectively. It also improves mean social welfare over proportional allocation by 6.475, 23.845, and 36.964. Although proportional allocation attains lower delay, it uses 54.8%, 129.1%, and 195.1% more computing power per active MD, which raises local costs and lowers social welfare. Overall, *the proposed method adapts effectively to real-world MD mobility. It achieves the highest social welfare and maintains a low maximum delay using substantially less computing power than proportional dispatch, striking a better balance*

Table 3Comparison of normalized fairness index under varying λ_0

Method	Task Arrival Rate λ_0										
	50	60	70	80	90	100	110	120	130	140	150
Proposed	0.9429	0.9552	0.9638	0.9702	0.9750	0.9787	0.9816	0.9840	0.9859	0.9875	0.9888
Non-Cooperation	0.7139	0.5702	0.4636	0.3876	0.3327	0.2918	0.2606	0.2361	0.2165	0.2005	0.1872

between delay reduction and computing-power costs in a dynamic environment.

Remark 1 (Time-Varying Wireless Channels). *The trace-driven experiments evaluate the proposed method under time-varying task arrivals and mobility-induced MD availability, but hold each active MD's transmission rate R_n fixed across epochs and thus do not capture the time-varying wireless channels. To incorporate dynamic wireless communication, MD n can estimate its channel gain $g_{n,k}$ at the beginning of epoch k , obtain $R_{n,k}$ from (2) with g_n replaced by $g_{n,k}$, and rerun the method using $s/R_{n,k}$. This extension assumes an approximately constant channel within each epoch. Faster fading or imperfect channel-state information would require shorter epochs or stochastic optimization extensions.*

7.4. Advantages of Cooperation over Non-cooperation

In Fig.6, we demonstrate the advantage of ESP-MD cooperation over non-cooperation. We compare our proposed method with the non-cooperation method in a scenario with 5 MDs and $\lambda_0 = 50$ tasks per second.

First, we show that ESP-MD cooperation leads to significant utility improvement. As illustrated in Fig.6a, the proposed method markedly improves the utilities of all MDs and the ESP. This increase in the MDs' utilities directly results from the higher rewards allocated to the MDs, as shown in Fig.6b. The underlying reason is that our NBS formulation incorporates both the ESP's and the MDs' utilities into a unified NBP. This enables the ESP to consider the MDs' utilities and offer higher rewards to them. In contrast, in the non-cooperation baseline, the ESP selfishly maximizes its own utility and offers lower rewards to the MDs. Consequently, it cannot incentivize the MDs to contribute more computing power to the system, as shown in Fig.6c. Therefore, our cooperative scheme leads to a win-win solution for the ESP and the MDs.

Second, we show that ESP-MD cooperation leads to enhanced fairness in both task dispatch rates and utilities. In Fig.6d, the proposed method achieves a more balanced distribution of task dispatch rates across all MDs compared to the non-cooperation baseline. Specifically, the proposed method yields a variance of 2.7 in task dispatch rates, compared to 9.7 for the non-cooperation baseline.

We further verify the enhanced fairness in utilities by comparing the normalized fairness index of the two methods under $N = 20$ MDs and vary λ_0 from 50 to 150 tasks per

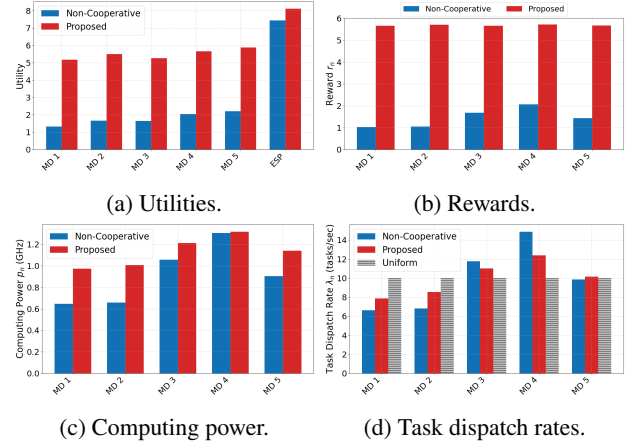


Figure 6: Comparison of the proposed method and the non-cooperation baseline.

second. The normalized fairness index $J(U_0, \{U_n\})$ is [21]:

$$J(U_0, \{U_n\}) = \frac{(U_0/\omega_0 + \sum_n U_n/\omega_n)^2}{(N+1) \left((U_0/\omega_0)^2 + \sum_n (U_n/\omega_n)^2 \right)} \quad (38)$$

where $J(U_0, \{U_n\}) \rightarrow 1$ represents the most fair, and $J(U_0, \{U_n\}) \rightarrow 1/(N+1)$ represents the most unfair. As shown in Table 3, the proposed method maintains an average normalized fairness index of 0.97, compared to 0.35 for the non-cooperation baseline. Therefore, our cooperative scheme is fairer in task dispatch rates and utilities.

Above results demonstrate that *ESP-MD cooperation significantly enhances both efficiency and fairness compared to non-cooperation.*

7.5. Sensitivity Analysis

7.5.1. Scalability of the Proposed Method

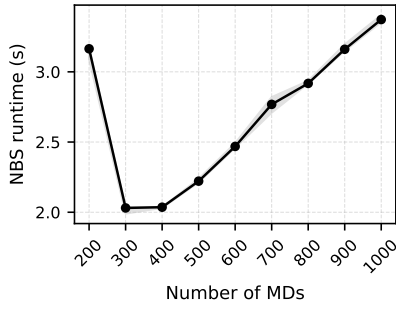
This subsection evaluates the scalability of the proposed method. We measure the runtime of the proposed method, the convergence iterations of the ADMM-based SWM stage and the subsequent ORD stage, and their cumulative communication costs.

The experiment varies the number of MDs from $N = 200$ to $N = 1000$ in increments of 100. We set $\lambda_0 = 2.5N$ tasks per second, yielding 500–2500 tasks per second across the tested sizes. We set all other parameters following Section 7.1. We repeat each set of experiment under five different random seeds. Runtime is measured on a MacBook Air with a 10-core Apple M4 processor and 16 GB of memory using Python 3.11, NumPy 2.4, and SciPy 1.17.

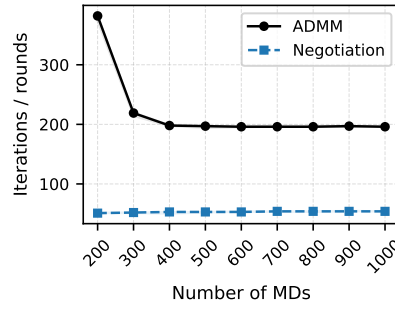
Table 4

Runtime, convergence behavior, and communication cost of the proposed method for different numbers of MDs. Each entry is the median of five runs.

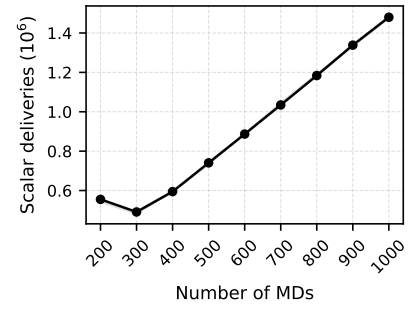
N	λ_0 (tasks/s)	Runtime (s)	ADMM iterations	Negotiation rounds	Scalar deliveries (10^6)
200	500	3.164	382	51	0.555
300	750	2.031	219	52	0.491
400	1000	2.036	198	53	0.594
500	1250	2.221	197	53	0.741
600	1500	2.469	196	53	0.887
700	1750	2.768	196	54	1.035
800	2000	2.918	196	54	1.184
900	2250	3.161	197	54	1.338
1000	2500	3.372	196	54	1.480



(a) End-to-end NBS runtime.



(b) Convergence iterations.



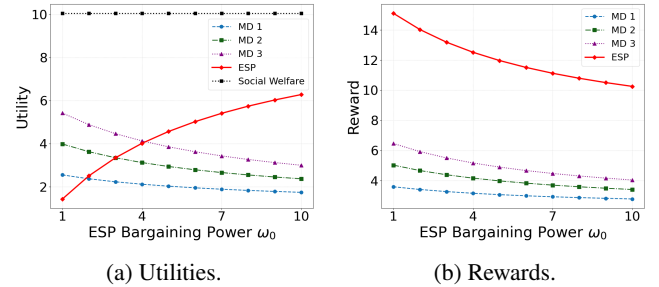
(c) Cumulative scalar deliveries.

Figure 7: Runtime, convergence behavior, and communication cost of the proposed method for different numbers of MDs. Lines show medians over five runs, and shaded regions show interquartile ranges.

Table 4 and Figs. 7a–7c report the runtime, convergence iterations, and cumulative communication cost for $N = 200$ –1000. As shown in Fig. 7a, the proposed method completes both NBS stages in a median of 3.372 seconds at $N = 1000$. The runtime decreases from $N = 200$ to $N = 300$ because the median number of ADMM iterations drops from 382 to 219. It then grows moderately from 2.036 to 3.372 seconds as N increases from 400 to 1000, while the ADMM iteration count remains between 196 and 198. In this stable-iteration range, a 2.5-fold increase in N increases the measured runtime by only 1.66 times.

The proposed method also maintains stable convergence as the system size increases. Fig. 7b shows that the median number of ADMM iterations remains between 196 and 198 for $N = 400$ –1000. The median number of negotiation rounds similarly remains between 53 and 54 over this range. Moreover, the measured runtime of the one-dimensional negotiation stage remains below 0.1 milliseconds at every tested scale. The ADMM-based SWM stage therefore accounts for nearly all of the computational cost.

The communication cost grows approximately linearly with the number of MDs. Each ADMM iteration delivers $7N$ scalars, and each negotiation round delivers $2N$ scalars. As shown in Fig. 7c, the median cumulative payload increases from 0.594 million scalars at $N = 400$ to 1.480 million scalars at $N = 1000$. This 2.49-fold increase closely matches the 2.5-fold increase in N because the iteration counts remain stable over this range. These results



(a) Utilities.

(b) Rewards.

Figure 8: Results of negotiation under varying ESP bargaining power.

demonstrate that *the proposed method scales effectively to 1000 eligible MDs under the tested load in terms of both computation and communication.*

7.5.2. Impacts of Bargaining Power

In Fig. 8, we investigate how bargaining power influences reward distribution in the negotiation process. We set $\lambda_0 = 30$ tasks per second and consider a scenario involving 3 identical MDs, each assigned a different bargaining power ($\omega_1 = 1, \omega_2 = 2, \omega_3 = 3$), while the ESP's bargaining power ω_0 varies from 1 to 10.

As shown in Fig. 8a, as the ESP's bargaining power increases, the ESP's utility increases while all MDs' utilities decrease. Meanwhile, the overall social welfare remains

unchanged. Notably, an MD with greater bargaining power consistently secures higher utility and a larger share of the reward in the negotiation. Fig. 8b further illustrates that as the ESP's bargaining power increases, both the total reward paid by the ESP and the individual rewards received by the MDs decrease. These results indicate that *the ESP and the MDs effectively divide the social welfare according to their respective bargaining powers—those with greater bargaining power obtain higher utility and rewards in the final outcome.*

8. Conclusion

In this paper, we present an efficient and fair cooperative computing power provision framework for edge systems. In this framework, the ESP collaborates with a set of self-interested MDs to serve the ESCs' delay-sensitive computation tasks. By utilizing the MDs' computing power, the ESP can flexibly handle the ESCs' demand surges instead of maintaining a costly, over-provisioned edge server cluster. To ensure sustainable cooperation, we formulate an NBP to model bargaining between the ESP and the MDs. The NBP is decomposed into a SWM problem and an ORD problem, solved respectively by a distributed ADMM algorithm and a negotiation algorithm. Notably, both algorithms require limited private-parameter exchange. Evaluations based on real-world datasets show that the proposed method achieves the highest social welfare with competitive delay under dynamic task arrivals and MD mobility. It also significantly enhances fairness in utilities and task dispatch rates compared to the non-cooperation baseline, and demonstrates effective scalability.

In future work, we will explore more general cooperative models involving multiple ESPs and dynamic MD participation. Also, integrating online learning techniques to handle real-time task arrivals and stochastic system dynamics is an interesting direction.

A. Proof of Proposition 1

In this appendix, we prove Proposition 1.

Under Assumption 1, the optimal social welfare $S^* > 0$, and the Nash product of the NBP in (11) is strictly positive and well-defined. Given a fixed, feasible point $(\lambda, \mathbf{p})$ in the domain of the SWM problem, we define the corresponding social welfare $S(\lambda, \mathbf{p}) \triangleq Q(D_{\max}(\lambda, \mathbf{p})) - \sum_{n \in \mathcal{N}} L_n(p_n)$. For a reward vector $\mathbf{r}$, we define $\delta_0 \triangleq Q(D_{\max}(\lambda, \mathbf{p})) - \sum_{r \in \mathcal{N}} r_n$, $\delta_n \triangleq r_n - L_n(p_n)$, $\forall n \in \mathcal{N}$, and $\delta = [\delta_0, \delta_1, \dots, \delta_N]$. Naturally, we have $\sum_{i \in \{0\} \cup \mathcal{N}} \delta_i = S(\lambda, \mathbf{p})$. Since the logarithmic objective is only well-defined when $\delta_i > 0, \forall i \in \mathcal{N} \cup \{0\}$, we restrict our analysis to this domain.

Given any feasible $\lambda, \mathbf{p}$, the NBP is equivalent to the following problem.

$$\max_{\delta} \sum_{i \in \mathcal{N} \cup \{0\}} \omega_i \log \delta_i \quad (39a)$$

$$\text{s.t.} \quad \sum_{i \in \mathcal{N} \cup \{0\}} \delta_i = S(\lambda, \mathbf{p}). \quad (39b)$$

The Lagrangian function of the problem in (39) is

$$\mathcal{L}(\delta, \mu) = - \sum_{i \in \mathcal{N} \cup \{0\}} \omega_i \log \delta_i + \mu \left(\sum_{i \in \mathcal{N} \cup \{0\}} \delta_i - S(\lambda, \mathbf{p}) \right). \quad (40)$$

Define $W \triangleq \sum_{i \in \mathcal{N} \cup \{0\}} \omega_i$. From the first-order optimality condition, we can derive

$$\mu^* = W / S(\lambda, \mathbf{p}), \quad (41)$$

$$\delta_i^* = \frac{\omega_i}{W} S(\lambda, \mathbf{p}), \forall i \in \mathcal{N} \cup \{0\}. \quad (42)$$

We define the optimal objective value of the problem in (39) as $V(\delta) \triangleq \sum_{i \in \mathcal{N} \cup \{0\}} \omega_i \log \delta_i$. By substituting (42) into $V(\delta)$, we can obtain the optimal objective value as follows.

$$V(\delta(\lambda, \mathbf{p})) = C + W \log S(\lambda, \mathbf{p}), \quad (43)$$

where $C = \sum_{i \in \mathcal{N} \cup \{0\}} \omega_i \log \omega_i - W \log W$ is a constant.

Consider the optimal solution to the NBP $(\lambda^{NBP}, \mathbf{p}^{NBP}, \mathbf{r}^{NBP})$. We will prove that $(\lambda^{NBP}, \mathbf{p}^{NBP})$ is optimal for the SWM problem by contradiction. Suppose that $(\lambda^{NBP}, \mathbf{p}^{NBP})$ is not optimal for the SWM problem, then there exists a feasible $(\lambda', \mathbf{p}')$ such that $S(\lambda', \mathbf{p}') > S(\lambda^{NBP}, \mathbf{p}^{NBP})$. For the pair of $(\lambda', \mathbf{p}')$, from (42), we have

$$\delta_i(\lambda', \mathbf{p}') = \frac{\omega_i}{W} S(\lambda', \mathbf{p}'), \forall i \in \mathcal{N} \cup \{0\}. \quad (44)$$

Given the values of $\delta_i(\lambda', \mathbf{p}')$ in (44), the optimal objective value of the problem in (39) is

$$\begin{aligned} V(\delta(\lambda', \mathbf{p}')) &= C + W \log S(\lambda', \mathbf{p}') \\ &> C + W \log S(\lambda^{NBP}, \mathbf{p}^{NBP}) = V(\delta(\lambda^{NBP}, \mathbf{p}^{NBP})). \end{aligned} \quad (45)$$

This contradicts the fact that $(\lambda^{NBP}, \mathbf{p}^{NBP}, \mathbf{r}^{NBP})$ is the optimal solution to the NBP. That is, $(\lambda^{NBP}, \mathbf{p}^{NBP}, \mathbf{r}^{NBP})$ must maximize $S(\lambda, \mathbf{p})$, i.e., be the optimal solution to the SWM problem.

Therefore, we conclude that the optimal task dispatch rates and computing power allocation for the NBP are the optimal solution to the SWM problem, i.e., the NBP maximizes the social welfare.

B. Proof of Proposition 2

In this appendix, we prove Proposition 2.

Given a fixed, feasible point $(\lambda, \mathbf{p})$ in the domain of the SWM problem, we define the corresponding social welfare $S(\lambda, \mathbf{p}) \triangleq Q(D_{\max}(\lambda, \mathbf{p})) - \sum_{n \in \mathcal{N}} L_n(p_n)$. For a reward vector $\mathbf{r}$, we define $\delta_0 \triangleq Q(D_{\max}(\lambda, \mathbf{p})) - \sum_{r \in \mathcal{N}} r_n$, and $\delta_n \triangleq r_n - L_n(p_n)$, $\forall n \in \mathcal{N}$. Naturally, we have $\sum_{i \in \{0\} \cup \mathcal{N}} \delta_i = S(\lambda, \mathbf{p})$. Since the logarithmic objective is only well-defined when $\delta_i > 0, \forall i \in \mathcal{N} \cup \{0\}$, we restrict our analysis to this domain.

With the fixed $\lambda, \mathbf{p}$, the NBP is equivalent to the following problem.

$$\max_{\delta_0, \delta_1, \dots, \delta_N} \prod_{i \in \mathcal{N} \cup \{0\}} (\delta_i)^{\omega_i} \quad (46a)$$

$$s.t. \quad \sum_{i \in \mathcal{N} \cup \{0\}} \delta_i = S(\lambda, \mathbf{p}). \quad (46b)$$

Define $x_i \triangleq \delta_i / \omega_i$ for $i \in \mathcal{N} \cup \{0\}$. The objective of the problem in (46) can be equivalently transformed as follows.

$$\begin{aligned} \prod_{i \in \mathcal{N} \cup \{0\}} (\delta_i)^{\omega_i} &= \prod_{i \in \mathcal{N} \cup \{0\}} (\omega_i x_i)^{\omega_i} \\ &= \left(\prod_{i \in \mathcal{N} \cup \{0\}} (\omega_i)^{\omega_i} \right) \prod_{i \in \mathcal{N} \cup \{0\}} (x_i)^{\omega_i}. \end{aligned} \quad (47)$$

The factor $\prod_{i \in \mathcal{N} \cup \{0\}} (\omega_i)^{\omega_i}$ is a constant. Hence, maximizing $\prod_{i \in \mathcal{N} \cup \{0\}} (\delta_i)^{\omega_i}$ is equivalent to maximizing $\prod_{i \in \mathcal{N} \cup \{0\}} (x_i)^{\omega_i}$.

Define $W \triangleq \sum_{i \in \mathcal{N} \cup \{0\}} \omega_i$, and define $\psi_i \triangleq \omega_i / W$, $\forall i \in \mathcal{N} \cup \{0\}$. Then, we have

$$\prod_{i \in \mathcal{N} \cup \{0\}} (x_i)^{\omega_i} = \left(\prod_{i \in \mathcal{N} \cup \{0\}} (x_i)^{\psi_i} \right)^W. \quad (48)$$

According to the weighted AM-GM inequality, we have

$$\prod_{i \in \mathcal{N} \cup \{0\}} (x_i)^{\psi_i} \leq \prod_{i \in \mathcal{N} \cup \{0\}} \psi_i x_i. \quad (49)$$

Combining equation (48) and inequality (49), we have the following inequation.

$$\begin{aligned} \prod_{i \in \mathcal{N} \cup \{0\}} (x_i)^{\omega_i} &= \left(\prod_{i \in \mathcal{N} \cup \{0\}} (x_i)^{\psi_i} \right)^W \\ &\leq \left(\sum_{i \in \mathcal{N} \cup \{0\}} \psi_i x_i \right)^W = \left(\frac{1}{W} \sum_{i \in \mathcal{N} \cup \{0\}} \omega_i x_i \right)^W \\ &= \left(\frac{S(\lambda, \mathbf{p})}{W} \right)^W. \end{aligned} \quad (50)$$

In (50), the equality holds if and only if all the x_i are equal, i.e., $x_0 = x_1 = \dots = x_N$. Meanwhile, the problem in (46) attains its optimum when the equality holds in (50).

Given the fixed $\lambda, \mathbf{p}$, we denote δ_i^* , $\forall i \in \mathcal{N} \cup \{0\}$ as the optimal solution to the problem in (46). And we denote $x_i^* = \delta_i^* / \omega_i$, $\forall i \in \mathcal{N} \cup \{0\}$. Since $\sum_{i \in \mathcal{N} \cup \{0\}} \omega_i x_i = S(\lambda, \mathbf{p})$ and all the x_i^* are equal, we have $x_i^* = S(\lambda, \mathbf{p}) / W$, $\forall i \in \mathcal{N} \cup \{0\}$. Therefore, we have

$$\delta_i^* = \frac{\omega_i}{W} S(\lambda, \mathbf{p}), \forall i \in \mathcal{N} \cup \{0\}. \quad (51)$$

Substituting the expression of δ_i^* in (51) into the objective function of the problem in (46) yields the optimal objective value:

$$\left(\prod_{i \in \mathcal{N} \cup \{0\}} (\omega_i)^{\omega_i} \right) \times \left(\frac{S(\lambda, \mathbf{p})}{W} \right)^W. \quad (52)$$

Note that, given any feasible $(\lambda, \mathbf{p})$, the optimal objective value in (52) is an increasing function of the social welfare $S(\lambda, \mathbf{p})$. Consider the optimal solution to the SWM problem $\lambda^*, \mathbf{p}^*$ and the optimal social welfare $S^* = S(\lambda^*, \mathbf{p}^*)$. We

have $S^* \geq S(\lambda, \mathbf{p})$ for all feasible $(\lambda, \mathbf{p})$. Therefore, $\lambda^*, \mathbf{p}^*$ maximizes the objective of the problem in (46) among all feasible $(\lambda, \mathbf{p})$.

Next, we show that, for the optimal solution to the SWM problem $(\lambda^*, \mathbf{p}^*)$, the solution to the ORD problem coincides with the optimal reward vector of the problem in (46). Recall that $\delta_0 = Q(D_{\max}(\lambda, \mathbf{p})) - \sum_{r \in \mathcal{N}} r_n$, and $\delta_n = r_n - L_n(p_n)$, $\forall n \in \mathcal{N}$. According to (51), given $(\lambda^*, \mathbf{p}^*)$, the optimal rewards $\mathbf{r}^*(\lambda^*, \mathbf{p}^*)$ of the problem in (46) satisfies

$$r_n^*(\lambda^*, \mathbf{p}^*) = L_n(p_n^*) + \frac{\omega_n}{W} S^*, \forall n \in \mathcal{N}. \quad (53)$$

We denote $Q^* = Q(D_{\max}(\lambda^*, \mathbf{p}^*))$ and $L_n^* = L_n(p_n^*)$. From the first-order optimality condition of the ORD problem, we have

$$-\frac{\omega_0}{Q^* - \sum_{m \in \mathcal{N}} r_m} + \frac{\omega_n}{r_n - L_n^*} = 0, \forall n \in \mathcal{N}. \quad (54)$$

It can be easily verified that $r_n^*(\lambda^*, \mathbf{p}^*)$ in (53) satisfies (54). One can also see that $r_n^*(\lambda^*, \mathbf{p}^*)$ in (53) is exactly the solution to the ORD problem in the discussion of Section 6. Therefore, the optimal solution to the ORD problem is equal to the optimal reward vector $\mathbf{r}^*(\lambda^*, \mathbf{p}^*)$ of the problem in (46).

In conclusion, the optimal solution $\lambda^*, \mathbf{p}^*$ to the SWM problem and the optimal solution $\mathbf{r}^*$ to the ORD problem are the optimal solution to the NBP, i.e., achieving the optimal objective value of the NBP. This finishes the proof.

C. Proof of Theorem 1

In this appendix, we prove Theorem 1 in two steps. First, we establish the equivalence between the SWM problem and the R-SWM problem. Second, we prove the objective convergence of the proposed ADMM algorithm on the R-SWM problem.

1) *Equivalence of SWM and R-SWM.* We first transform the SWM problem into the T-SWM problem by introducing an auxiliary variable $D_{\max}$ and incorporating constraints (14) and (15). If $\lambda^*, \mathbf{p}^*$ is an optimal solution to the SWM problem, then defining $D_{\max}^* = \max_{n \in \mathcal{N}} D_n(\lambda_n^*, p_n^*)$ yields a triplet $(\lambda^*, \mathbf{p}^*, D_{\max}^*)$ that is a feasible solution to the T-SWM problem and attains the same optimal objective value. If $(\lambda^*, \mathbf{p}^*, D_{\max}^*)$ is a solution to the T-SWM problem, then we have $D_n(\lambda_n^*, p_n^*) \leq D_{\max}^* \leq D_0 - \varepsilon_2$, $\forall n \in \mathcal{N}$ according to constraints (14) and (15). Hence, $(\lambda^*, \mathbf{p}^*)$ is feasible for the SWM problem. We can further derive that $\max_{n \in \mathcal{N}} D_n(\lambda_n^*, p_n^*) = D_{\max}^*$, because function $Q(\cdot)$ monotonically increases as $D_{\max}$ decreases, thus we have at least one binding constraint in constraints (15) at the optimum of the T-SWM problem. Hence, $(\lambda^*, \mathbf{p}^*)$ is also optimal for the SWM problem. Therefore, the SWM problem and the T-SWM problem are equivalent.

We next transform the T-SWM problem into the R-SWM problem by introducing auxiliary variables $\hat{\lambda}, \hat{D}_{\max}$. Since these variables are the exact copies of $\lambda, D_{\max}$, the T-SWM

problem and the R-SWM problem are exactly equivalent. Consequently, the SWM problem and the R-SWM problem are equivalent.

2) Objective Convergence of the Proposed Algorithm.

We now prove that Algorithm 1 converges to the optimal objective value of the R-SWM problem as the number of iterations tends to infinity.

Before we proceed, we cast the R-SWM problem into the standard two-block ADMM formulation [6]:

$$\min_{x,z} f(x) + g(z) \quad (55a)$$

$$s.t. Ax + Bz = c \quad (55b)$$

Specifically, we define $x \triangleq (\lambda, D_{\max})$, $z \triangleq (p, \hat{\lambda}, \hat{D}_{\max})$. Constraints (17) and (18) can be rewritten as $Ax + Bz = 0$ by properly defining matrices A and B . Constraints (10b) and (14) define a non-empty closed convex set $\mathcal{X}$ and can be incorporated into the objective function via an indicator function $I_{\mathcal{X}}(x)$, where $I_{\mathcal{X}}(x) = 0$ if $x \in \mathcal{X}$ otherwise $I_{\mathcal{X}}(x) = \infty$. Constraints (10d), (19), and (20) define a non-empty closed convex set $\mathcal{Z}$ and can be incorporated into the objective function via the indicator function $I_{\mathcal{Z}}(z)$. We can thus define the two extended-real-valued functions $f(x) = -Q(D_{\max}) + I_{\mathcal{X}}(x)$, $g(z) = \sum_{n \in \mathcal{N}} L_n(p_n) + I_{\mathcal{Z}}(z)$. The unaugmented Lagrangian is given by

$$\mathcal{L}(x, z, y) = f(x) + g(z) + y^T(Ax + Bz), \quad (56)$$

where y is the dual variable associated with the constraint $Ax + Bz = 0$.

As stated in [6], the convergence of Algorithm 1 relies on two conditions: 1) the extended-real-valued functions $f(x)$ and $g(z)$ are closed, proper, and convex, 2) the unaugmented Lagrangian $\mathcal{L}$ has a saddle point. We will show that these two conditions are both satisfied.

The extended-real-valued functions $f(x)$ and $g(z)$ are indeed closed, proper, and convex, because they are the sum of closed, proper, convex functions.

- Function $-Q(D_{\max})$ is convex due to the concavity of $Q(\cdot)$. It's closed because its domain $[0, D_0 - \varepsilon_2]$ is closed and it's a continuous function [7]. It is proper because it is finite on its domain.
- Each term $L_n(p_n)$ is convex. It is continuous and finite on the closed interval $[0, F_n]$, so it's closed and proper. The sum of the terms $\sum_{n \in \mathcal{N}} L_n(p_n)$ are therefore closed, proper and convex.
- Given a non-empty, closed, convex set $C \subseteq \mathbb{R}^m$, the associated indicator function I_C is convex. It is also proper because there exists $c_0 \in C$ such that $I_C(c_0) = 0$ and it's never $-\infty$ by definition. The epigraph of I_C is $(C \times [0, +\infty)) \cup ((\mathbb{R}^m \setminus C) \times \{+\infty\})$, which is the union of two closed set. Hence, the epigraph of I_C is a closed set. Consequently, the function I_C is closed [7]. Since $\mathcal{X}$ and $\mathcal{Z}$ are non-empty, closed, convex sets, the associated indicator functions $I_{\mathcal{X}}$ and $I_{\mathcal{Z}}$ are closed, proper, and convex.

The unaugmented Lagrangian $\mathcal{L}$ indeed has a saddle point. We prove that strong duality holds for the R-SWM problem. Since all constraints except (20) are either equality or affine inequality constraints, we can examine the weak Slater condition [7], which only requires every non-affine inequality constraint to be strict. We aim to show that there exists a feasible point $(\lambda^0, p^0, D_{\max}^0, \hat{\lambda}^0, \hat{D}_{\max}^0)$ for the R-SWM problem such that $D_n(\hat{\lambda}_n^0, p_n^0) < \hat{D}_{\max, n}^0, \forall n \in \mathcal{N}$. Consider a feasible point $(\lambda^0, p^0, D_{\max}^0, \hat{\lambda}^0, \hat{D}_{\max}^0)$ such that $D_n(\hat{\lambda}_n^0, p_n^0) = \hat{D}_{\max, n}^0$ for some $n \in \mathcal{N}$. We can increase $D_{\max}^0$ by a small constant $\delta > 0$ such that $D_{\max}^0 + \delta \leq D_0 - \varepsilon_2$, then we enforce $\hat{D}_{\max, n} = D_{\max}^0, \forall n \in \mathcal{N}$. Therefore, we have $D_n(\hat{\lambda}_n^0, p_n^0) < \hat{D}_{\max, n}^0 + \delta$ and maintain feasibility. We argue that such a construction can always be performed as long as ε_2 is small enough. Hence, the R-SWM problem satisfies the weak Slater's condition, which implies strong duality. Therefore, the unaugmented Lagrangian $\mathcal{L}$ has a saddle point.

Consequently, because the two conditions stated in [6] are satisfied, the iterates of Algorithm 1 satisfy objective convergence, i.e., the generated objective values $\{S^{[l]}\}$ of the iterates approach the optimal value S^* as $t \rightarrow \infty$. The proof is thus finished.

D. Feasibility of Assumption 1

In this appendix, we further explain the rationality of Assumption 1 and characterize when it holds or fails. It specifies the scenarios where the ESP-MD cooperation creates a positive social surplus and the Nash bargaining problem in (11) is economically meaningful.

For any feasible task dispatch and computing power allocation (λ, p) , recall that we have

$$L_n(p_n) = E_n(p_n) + b_n(0) - b_n(p_n). \quad (57)$$

The excess utility of the ESP is

$$U_0 - \phi_0 = Q(D_{\max}) - \sum_{n \in \mathcal{N}} r_n, \quad (58)$$

and the excess utility of MD n is

$$U_n - \phi_n = r_n - L_n(p_n). \quad (59)$$

Therefore, the aggregate excess utility of all participants is

$$(U_0 - \phi_0) + \sum_{n \in \mathcal{N}} (U_n - \phi_n) = Q(D_{\max}) - \sum_{n \in \mathcal{N}} L_n(p_n) \triangleq S(\lambda, p). \quad (60)$$

The reward terms cancel out because they are monetary transfers from the ESP to the MDs. Thus, the social welfare measures the net surplus created by cooperation.

Lemma 1. For any feasible (λ, p) , there exists a reward vector r such that

$$U_0 - \phi_0 > 0, \quad U_n - \phi_n > 0, \quad \forall n \in \mathcal{N}, \quad (61)$$

if and only if $S(\lambda, p) > 0$.

Proof. If there exists a reward vector such that all participants obtain positive excess utilities, then we naturally have

$$S(\lambda, p) = (U_0 - \phi_0) + \sum_{n \in \mathcal{N}} (U_n - \phi_n) > 0. \quad (62)$$

Conversely, suppose that $S(\lambda, p) > 0$. Choose positive numbers $x_0 > 0$ and $x_n > 0, \forall n \in \mathcal{N}$, such that

$$x_0 + \sum_{n \in \mathcal{N}} x_n = S(\lambda, p). \quad (63)$$

Set

$$r_n = L_n(p_n) + x_n, \quad \forall n \in \mathcal{N}. \quad (64)$$

Since $L_n(p_n) \geq 0$, we have $r_n \geq 0$. Moreover, we have

$$U_n - \phi_n = r_n - L_n(p_n) = x_n > 0, \quad \forall n \in \mathcal{N}, \quad (65)$$

and

$$U_0 - \phi_0 = Q(D_{\max}) - \sum_{n \in \mathcal{N}} r_n = S(\lambda, p) - \sum_{n \in \mathcal{N}} x_n = x_0 > 0. \quad (66)$$

Thus, all participants can obtain positive excess utilities. $\square$

The above lemma shows that Assumption 1 is necessary and sufficient for the existence of a strictly individually rational reward distribution. If $S^* \leq 0$, then the cooperation does not create a positive social surplus. In that case, rewards can only redistribute utility among participants but cannot make the ESP and all MDs strictly better off simultaneously.

Next, we characterize a threshold condition when Assumption 1 holds. Consider a target system-wide delay upper bound D satisfying $0 < D \leq D_0 - \epsilon_2$, where ϵ_2 is the small positive constant introduced in constraint (10e) of the NBP in (11). For MD n , its delay should be smaller than the target delay upper bound, that is,

$$D_n(\lambda_n, p_n) = T_n^t + \frac{1}{p_n/(sl) - \lambda_n} \leq D. \quad (67)$$

This requires $D > T_n^t$ and

$$\frac{p_n}{sl} - \lambda_n \geq \frac{1}{D - T_n^t}. \quad (68)$$

Combining this with the queue stability constraint in (10c) of the NBP, which is

$$\lambda_n \leq \frac{p_n}{sl} - \epsilon_1, \quad (69)$$

we define

$$\eta_n(D) \triangleq \max \left\{ \epsilon_1, \frac{1}{D - T_n^t} \right\}. \quad (70)$$

Then the minimum computing power required by MD n to process task rate λ_n under delay target D is

$$p_n^{\min}(\lambda_n, D) = sl(\lambda_n + \eta_n(D)). \quad (71)$$

Therefore, the target delay D is feasible if there exist non-negative dispatch rates $\{\lambda_n\}_{n \in \mathcal{N}}$ such that

$$\sum_{n \in \mathcal{N}} \lambda_n = \lambda_0, \quad (72)$$

and

$$sl(\lambda_n + \eta_n(D)) \leq F_n, \quad \forall n \in \mathcal{N}. \quad (73)$$

Equivalently, for the MD $n \in \mathcal{N}$, feasibility under D requires

$$D > T_n^t, \quad \frac{F_n}{sl} \geq \eta_n(D), \quad \forall n \in \mathcal{N}, \quad (74)$$

and

$$\sum_{n \in \mathcal{N}} \left(\frac{F_n}{sl} - \eta_n(D) \right) \geq \lambda_0. \quad (75)$$

For any given D and λ_n , the MD $n \in \mathcal{N}$ contributes the minimum-cost computing power allocation $p_n^{\min}(\lambda_n, D) = sl(\lambda_n + \eta_n(D))$. Since $L_n(p_n)$ is increasing in p_n , we define the minimum aggregate MD cost under delay target D as

$$C(D) = \min_{\lambda} \sum_{n \in \mathcal{N}} L_n(sl(\lambda_n + \eta_n(D))), \quad (76)$$

subject to

$$\sum_{n \in \mathcal{N}} \lambda_n = \lambda_0, \quad (77)$$

$$0 \leq \lambda_n \leq \frac{F_n}{sl} - \eta_n(D), \quad \forall n \in \mathcal{N}. \quad (78)$$

Then the optimal social welfare can be characterized as

$$S^* = \max_D \left\{ \bar{Q} - \frac{o}{D_0 - D} - C(D) \right\}, \quad (79)$$

where the maximization is taken over all feasible delay targets $D \leq D_0 - \epsilon_2$. Hence, Assumption 1 holds if and only if there exists a feasible delay target D such that

$$\bar{Q} - \frac{o}{D_0 - D} > C(D). \quad (80)$$

Equivalently, the ideal service revenue $\bar{Q}$ must satisfy the revenue-threshold condition

$$\bar{Q} > \min_D \left\{ \frac{o}{D_0 - D} + C(D) \right\}. \quad (81)$$

This characterization gives a clear practical interpretation. Assumption 1 holds when the ESP's QoS revenue from serving delay-sensitive tasks is sufficiently large relative to the MDs' aggregate energy and opportunity costs. It is more likely to hold when the task service has high value, the delay tolerance D_0 is not overly stringent, the wireless transmission delays T_n^t are small, the MDs have sufficient

computing capacities F_n , and the costs represented by $L_n(p_n)$ are low.

On the other hand, the assumption may fail in two typical cases. First, the feasibility region may be empty, e.g., when the task arrival rate λ_0 is too high, the MDs' computing capacities are insufficient, the wireless links are poor, or the delay requirement is too stringent. Second, even if feasible task dispatch and computing power allocation exist, the generated revenue may still be no larger than the aggregate MD costs, i.e.,

$$\bar{Q} - \frac{o}{D_0 - D} \leq C(D) \quad (82)$$

for all feasible D . In such cases, cooperation does not generate a positive social surplus, and rational participants would not engage in the bargaining process.

E. Baseline Implementation Details

In this appendix, we present the implementation details for the baseline methods. All baselines use the same system model as specified in Section 3.

E.1. Non-Cooperation

In the non-cooperation baseline, we formulate the strategic interaction between the ESP and the MDs as a Stackelberg game. The ESP acts as the leader and announces a unit computing-power price π . It also determines the task-dispatch rates λ and a system-wide delay bound $D_{\max}$. Each MD then selects its computing-power contribution to maximize its payment minus its local computing cost.

For a given $(\lambda_n, D_{\max})$, the minimum computing power that allows MD n to meet the delay bound is

$$\underline{p}_n(\lambda_n, D_{\max}) = s\lambda_n + \frac{s\lambda}{D_{\max} - s/R_n}. \quad (83)$$

The follower problem of MD n is

$$p_n^*(\pi, \lambda_n, D_{\max}) \in \arg \max_{p_n} \pi p_n - L_n(p_n) \quad (84a)$$

$$\text{s.t. } \underline{p}_n(\lambda_n, D_{\max}) \leq p_n \leq F_n, \quad (84b)$$

$$\pi p_n - L_n(p_n) \geq 0. \quad (84c)$$

The last constraint represents individual rationality in the non-cooperative transaction. Anticipating the MD responses, the ESP solves

$$\max_{\pi, \lambda, D_{\max}} Q(D_{\max}) - \pi \sum_{n \in \mathcal{N}} p_n^*(\pi, \lambda_n, D_{\max}) \quad (85a)$$

$$\text{s.t. } \sum_{n \in \mathcal{N}} \lambda_n = \lambda_0, \quad (85b)$$

$$\lambda_n \geq 0, \quad \forall n \in \mathcal{N}, \quad (85c)$$

$$\underline{p}_n(\lambda_n, D_{\max}) \leq F_n, \quad \forall n \in \mathcal{N}, \quad (85d)$$

$$\max_{n \in \mathcal{N}} \frac{s}{R_n} < D_{\max} < D_0. \quad (85e)$$

After solving the game, the reward paid to MD n is $r_n^{\text{NC}} = \pi^* p_n^*$.

We reformulate the bi-level optimization problem of the Stackelberg game as a single-level optimization problem. We first use the KKT conditions of the MD's follower problem in (84) to characterize the best response of each MD. Let ξ denote the Lagrangian multipliers of the follower problems. Let $\mathbf{z} = (\pi, \lambda, D_{\max}, \mathbf{p}, \xi)$ collect the primal and auxiliary variables, and let $\mathbf{c}_{n,\mu}(\mathbf{z})$ collect the stationarity residual and the smoothed complementarity residuals of MD n . The resulting single-level optimization problem is

$$\min_{\mathbf{z}} - \left[Q(D_{\max}) - \pi \sum_{n \in \mathcal{N}} p_n \right] + \rho \sum_{n \in \mathcal{N}} \|\mathbf{c}_{n,\mu}(\mathbf{z})\|_2^2 \quad (86a)$$

$$\text{s.t. } \sum_{n \in \mathcal{N}} \lambda_n = \lambda_0, \quad (86b)$$

$$\lambda_n \geq 0, \quad \forall n \in \mathcal{N}, \quad (86c)$$

$$\underline{p}_n(\lambda_n, D_{\max}) \leq p_n \leq F_n, \quad \forall n \in \mathcal{N}, \quad (86d)$$

$$\max_{n \in \mathcal{N}} \frac{s}{R_n} < D_{\max} < D_0, \quad (86e)$$

$$\pi \geq 0, \quad \xi \geq 0. \quad (86f)$$

Here, $\rho > 0$ is the penalty coefficient. The residual vector is

$$\mathbf{c}_{n,\mu}(\mathbf{z}) = \begin{bmatrix} q_n(\mathbf{z}) \\ \Phi_\mu(\xi_{n,1}(\mathbf{z}), b_{n,1}(\mathbf{z})) \\ \Phi_\mu(\xi_{n,2}(\mathbf{z}), b_{n,2}(\mathbf{z})) \\ \Phi_\mu(\xi_{n,3}(\mathbf{z}), b_{n,3}(\mathbf{z})) \end{bmatrix}, \quad (87)$$

where $q_n(\mathbf{z})$ is the stationarity condition of the follower problem. The three pairs $(\xi_{n,j}, b_{n,j})$, $j = 1, 2, 3$, represent the complementary slackness conditions of constraints (84b)-(84c), respectively. Each complementary slackness is smoothed using the Fischer-Burmeister function:

$$\Phi_\mu(\xi, b) = \sqrt{\xi^2 + b^2 + \mu^2} - \xi - b \quad (88)$$

The constrained nonlinear problem is solved using SLSQP in a continuation procedure, where the solution of one continuation step initializes the next step. The smoothing parameter in (88) is set as $\mu^{(t)} = 10^{-6}(0.8)^t$, where t is the current iteration. The penalty coefficient is fixed at $\rho = 10^2$. At each continuation step, SLSQP uses an objective tolerance of 10^{-5} and at most 2000 iterations. The initial point is $\pi = 1$, $\lambda_n = \lambda_0/N$, $D_{\max} = (\max_n \{s/R_n\} + 10^{-3} + D_0)/2$, and $p_n = \min\{F_n/2, 0.5\}$, with all auxiliary multipliers initialized to zero.

E.2. Contract-based method

In the contract-based baseline, we formulate the ESP as a principal and the MDs as agents under asymmetric information. The MD population is represented by a finite type set $\mathcal{K} = \{1, \dots, K\}$, and N_k denotes the number of MDs of type k . Each type is characterized by its computing capacity, wireless rate, and local computing cost. Specifically, we set $K = 5$ in this baseline. The MDs are first sorted

in ascending order of their energy-consumption coefficient c_n . We then select K representative MDs that are evenly spaced along the sorted list. To determine how many actual MDs each representative stands for, we divide the sorted list into K segments of roughly equal size. The number of MDs in each segment defines the population count N_k for the corresponding representative type.

The ESP designs a contract menu $\mathcal{C} = \{(\lambda_k, p_k, r_k)\}_{k \in \mathcal{K}}$. Each item specifies a task rate, a computing-power contribution, and a reward. The optimal-contract-design problem is

$$\max_{\mathcal{C}, D_{\max}} Q(D_{\max}) - \sum_{k \in \mathcal{K}} N_k r_k \quad (89a)$$

$$\text{s.t.} \quad \sum_{k \in \mathcal{K}} N_k \lambda_k = \lambda_0, \quad (89b)$$

$$0 \leq p_k \leq F_k, \quad \forall k \in \mathcal{K}, \quad (89c)$$

$$\lambda_k \leq \frac{p_k}{sl} - \varepsilon_1, \quad \forall k \in \mathcal{K}, \quad (89d)$$

$$\frac{s}{R_k} + \frac{1}{p_k/(sl) - \lambda_k} \leq D_{\max}, \quad \forall k \in \mathcal{K}, \quad (89e)$$

$$D_{\max} \leq D_0 - \varepsilon_2, \quad (89f)$$

$$r_k - L_k(p_k) \geq 0, \quad \forall k \in \mathcal{K}, \quad (89g)$$

$$r_k - L_k(p_k) \geq r_j - L_k(p_j), \quad \forall k \in \mathcal{K}, \quad j \in \mathcal{A}_k. \quad (89h)$$

Constraint (89g) is the individual-rationality constraint. Constraint (89h) is the incentive-compatibility constraint. It ensures that type k weakly prefers its designated contract to every alternative that it can feasibly execute.

After the menu is announced, an MD of type k selects

$$j^*(k) \in \arg \max_{j \in \mathcal{A}_k} \{r_j - L_k(p_j)\}. \quad (90)$$

The selected contract item is implemented directly. An MD of type k is assigned task rate λ_k , contributes computing power p_k , and receives reward r_k .

E.3. Proportional Dispatch

In proportional dispatch, the ESP dispatches the tasks proportional to each MD's computing capacities, *i.e.*,

$$\lambda_n^P = \lambda_0 \frac{F_n}{\sum_{i \in \mathcal{N}} F_i}, \quad \forall n \in \mathcal{N}. \quad (91)$$

Each MD randomly selects a feasible computing power that satisfies the delay constraint D_0 . This random computing-power selection of each MD aims to improve the computing power provision in this baseline implementation. If each MD acts selfishly in choosing its computing power, it will supply only the minimum computing power needed to meet the delay constraint. The random computing-power selection hence improves the performance of this baseline.

In our implementation, each MD n randomly select a target delay $\tilde{D}_n^P < D_0$. Its computing power is obtained by

inverting the delay expression:

$$p_n^P = sl \lambda_n^P + \frac{sl}{\tilde{D}_n^P - s/R_n} \leq F_n. \quad (92)$$

The ESP collects each MD n 's computing power p_n^P and computes the realized delay and the resulting social welfare $S(\lambda^P, p^P)$. If the social welfare is positive, it would be distributed to each MD in proportion to its computing capacity:

$$r_n^P = [S(\lambda^P, p^P)]^+ \frac{F_n}{\sum_{i \in \mathcal{N}} F_i}, \quad (93)$$

where $[x]^+ = \max\{x, 0\}$.

E.4. Uniform Dispatch

In proportional dispatch, the ESP dispatches the tasks uniformly to each MD:

$$\lambda_n^U = \frac{\lambda_0}{N}, \quad \forall n \in \mathcal{N}. \quad (94)$$

Each MD randomly selects a feasible computing power that satisfies the delay constraint D_0 . This random computing-power selection of each MD aims to improve the computing power provision in this baseline implementation. If each MD acts selfishly in choosing its computing power, it will supply only the minimum computing power needed to meet the delay constraint. The random computing-power selection hence improves the performance of this baseline.

In our implementation, each MD n randomly selects a target delay $\tilde{D}_n^U < D_0$, and its computing power is determined by:

$$p_n^U = sl \lambda_n^U + \frac{sl}{\tilde{D}_n^U - s/R_n} \leq F_n. \quad (95)$$

The ESP collects each MD n 's computing power p_n^U and computes the realized delay and the resulting social welfare $S(\lambda^U, p^U)$. If the social welfare is positive, it would be distributed to each MD uniformly:

$$r_n^U = \frac{[S(\lambda^U, p^U)]^+}{N}. \quad (96)$$

CRedit authorship contribution statement

Beiyuan Liu: Formal analysis, Methodology, Software, Visualization, Writing – original draft, Writing – review and editing. **Yifei Wu:** Formal analysis, Methodology, Software, Visualization, Writing – original draft, Writing – review and editing. **Guocheng Liao:** Conceptualization, Formal analysis, Methodology, Funding acquisition, Supervision, Writing – review and editing. **Jianguo Chen:** Investigation, Validation. **Qian Ma:** Investigation, Validation. **Xu Chen:** Investigation, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62202512, Grant 62372481, and Grant 62372486, in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515011173, in part by the Guangxi Key Research and Development Program under Grant AB24010160, in part by the Shenzhen Science and Technology Program under Grant ZDCY20250901112501002, and in part by the Pearl River Talent Plan under Grant 2023QN10X579.

Data availability

Data will be made available on request.

References

- [1] Abbas, N., Zhang, Y., Taherkordi, A., Skeie, T., 2018. Mobile edge computing: A survey. *IEEE Internet of Things Journal* 5, 450–465. doi:10.1109/JIOT.2017.2750180.
- [2] Albin, S.L., 1982. On poisson approximations for superposition arrival processes in queues. *Management Science* 28, 126–137. URL: <https://ideas.repec.org/a/inm/ormnsc/v28y1982i2p126-137.html>, doi:10.1287/mnsc.28.2.126.
- [3] Apostolopoulos, P.A., Fragkos, G., Tsiropoulou, E.E., Papavassiliou, S., 2023. Data offloading in uav-assisted multi-access edge computing systems under resource uncertainty. *IEEE Transactions on Mobile Computing* 22, 175–190. doi:10.1109/TMC.2021.3069911.
- [4] Apple Inc., 2024. Apple introduces m4 chip. URL: <https://www.apple.com/newsroom/2024/05/apple-introduces-m4-chip/>.
- [5] Boche, H., Schubert, M., 2009. Nash bargaining and proportional fairness for wireless systems. *IEEE/ACM Transactions on Networking* 17, 1453–1466.
- [6] Boyd, S., Parikh, N., Chu, E., Peleato, B., Eckstein, J., et al., 2011. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends® in Machine Learning* 3, 1–122.
- [7] Boyd, S.P., Vandenberghe, L., 2004. *Convex optimization*. Cambridge university press.
- [8] Cao, X., Tang, G., Guo, D., Li, Y., Zhang, W., 2020. Edge federation: Towards an integrated service provisioning model. *IEEE/ACM Transactions on Networking* 28, 1116–1129. doi:10.1109/TNET.2020.2979361.
- [9] Cha, C., Park, H., 2022. Ecs-nbs: Exact computation of sequential nash bargaining solutions. *IEEE Transactions on Vehicular Technology* 71, 13453–13457. doi:10.1109/TVT.2022.3196063.
- [10] Chen, L., Zhou, S., Xu, J., 2018. Computation peer offloading for energy-constrained mobile edge computing in small-cell networks. *IEEE/ACM transactions on networking* 26, 1619–1632.
- [11] Chen, W.Y., Chou, P.Y., Wang, C.Y., Hwang, R.H., Chen, W.T., 2023. Dual pricing optimization for live video streaming in mobile edge computing with joint user association and resource management. *IEEE Transactions on Mobile Computing* 22, 858–873. doi:10.1109/TMC.2021.3089229.
- [12] Chen, X., 2014. Decentralized computation offloading game for mobile cloud computing. *IEEE Transactions on Parallel and Distributed Systems* 26, 974–983.
- [13] Chen, X., Jiao, L., Li, W., Fu, X., 2015. Efficient multi-user computation offloading for mobile-edge cloud computing. *IEEE/ACM transactions on networking* 24, 2795–2808.
- [14] Chen, X., Zhu, G., Ding, H., Zhang, L., Zhang, H., Fang, Y., 2022. End-to-end service auction: A general double auction mechanism for edge computing services. *IEEE/ACM Transactions on Networking* 30, 2616–2629. doi:10.1109/TNET.2022.3179239.
- [15] Chu, W., Jia, X., Yu, Z., Lui, J.C.S., Lin, Y., 2023. Joint service caching, resource allocation and task offloading for mec-based networks: A multi-layer optimization approach. *IEEE Transactions on Mobile Computing* 23, 2958–2975.
- [16] Dai, X., Xiao, Z., Jiang, H., Lui, J.C.S., 2024. Uav-assisted task offloading in vehicular edge computing networks. *IEEE Transactions on Mobile Computing* 23, 2520–2534. doi:10.1109/TMC.2023.3259394.
- [17] Guo, Y., Wang, S., Zhou, A., Xu, J., Yuan, J., Hsu, C.H., 2020. User allocation-aware edge cloud placement in mobile edge computing. *Software: Practice and Experience* 50, 489–502. doi:10.1002/spe.2685.
- [18] Hao, H., Xu, C., Zhang, W., Yang, S., Muntean, G.M., 2024. Joint task offloading, resource allocation, and trajectory design for multi-uav cooperative edge computing with task priority. *IEEE Transactions on Mobile Computing* 23, 8649–8663. doi:10.1109/TMC.2024.3350078.
- [19] He, B., Yuan, X., 2015. On non-ergodic convergence rate of douglas–rachford alternating direction method of multipliers. *Numerische Mathematik* 130, 567–577.
- [20] Iosifidis, G., Gao, L., Huang, J., Tassioulas, L., 2017. Efficient and fair collaborative mobile internet access. *IEEE/ACM Transactions on Networking* 25, 1386–1400. doi:10.1109/TNET.2016.2638939.
- [21] Jain, R.K., Chiu, D.M.W., Hawe, W.R., et al., 1984. A quantitative measure of fairness and discrimination. *Eastern Research Laboratory, Digital Equipment Corporation, Hudson, MA* 21, 2022–2023.
- [22] Jamil, B., Ijaz, H., Shojafar, M., Munir, K., Buyya, R., 2022. Resource allocation and task scheduling in fog computing and internet of everything environments: A taxonomy, review, and future directions. *ACM Comput. Surv.* 54. URL: <https://doi.org/10.1145/3513002>, doi:10.1145/3513002.
- [23] Jiang, H., Dai, X., Xiao, Z., Iyengar, A., 2023. Joint task offloading and resource allocation for energy-constrained mobile edge computing. *IEEE Transactions on Mobile Computing* 22, 4000–4015. doi:10.1109/TMC.2022.3150432.
- [24] Lee, C.Y., Lin, C.H., Chang, Z.L., Wang, C.Y., Wei, H.Y., 2024. Joint resource allocation and intrusion prevention system deployment for edge computing. *IEEE Transactions on Services Computing* 17, 2502–2515.
- [25] Li, C., Wang, S., Huang, X., Li, X., Yu, R., Zhao, F., 2019. Parked vehicular computing for energy-efficient internet of vehicles: A contract theoretic approach. *IEEE Internet of Things Journal* 6, 6079–6088. doi:10.1109/JIOT.2018.2869892.
- [26] Li, G., Cai, J., Chen, X., Su, Z., 2023. Nonlinear online incentive mechanism design in edge computing systems with energy budget. *IEEE Transactions on Mobile Computing* 22, 4086–4102. doi:10.1109/TMC.2022.3148034.
- [27] Li, Y., Li, F., Yang, S., Zhang, C., Zhu, L., Wang, Y., 2024. A cooperative analysis to incentivize communication-efficient federated learning. *IEEE Transactions on Mobile Computing* 23, 10175–10190. doi:10.1109/TMC.2024.3373501.
- [28] Li, Y., Yang, B., Wu, H., Han, Q., Chen, C., Guan, X., 2022a. Joint offloading decision and resource allocation for vehicular fog-edge computing networks: A contract-stackelberg approach. *IEEE Internet of Things Journal* 9, 15969–15982. doi:10.1109/JIOT.2022.3150955.
- [29] Li, Y., Zhou, A., Ma, X., Wang, S., 2022b. Profit-aware edge server placement. *IEEE Internet of Things Journal* 9, 55–67.
- [30] Liu, L., Chang, Z., Guo, X., Mao, S., Ristaniemi, T., 2018. Multi-objective optimization for computation offloading in fog computing. *IEEE Internet of Things Journal* 5, 283–294. doi:10.1109/JIOT.2017.2780236.
- [31] Liu, L., Li, H., Gruteser, M., 2019. Edge assisted real-time object detection for mobile augmented reality, in: *The 25th Annual International Conference on Mobile Computing and Networking*. doi:10.1145/3300061.3300116.
- [32] Liu, L., Zhao, M., Yu, M., Jan, M.A., Lan, D., Taherkordi, A., 2023a. Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks. *IEEE Transactions on Intelligent Transportation Systems* 24, 2169–2182. doi:10.1109/TITS.2022.3142566.

- [33] Liu, T., Guo, D., Xu, Q., Gao, H., Zhu, Y., Yang, Y., 2023b. Joint task offloading and dispatching for mec with rational mobile devices and edge nodes. *IEEE Transactions on Cloud Computing* 11, 3262–3273.
- [34] Luo, Q., Li, C., Luan, T.H., Shi, W., 2022. Minimizing the delay and cost of computation offloading for vehicular edge computing. *IEEE Transactions on Services Computing* 15, 2897–2909. doi:10.1109/TSC.2021.3064579.
- [35] Ma, L., Wang, X., Wang, X., Wang, L., Shi, Y., Huang, M., 2022. Tcda: Truthful combinatorial double auctions for mobile edge computing in industrial internet of things. *IEEE Transactions on Mobile Computing* 21, 4125–4138. doi:10.1109/TMC.2021.3064314.
- [36] Mao, Y., Zhang, J., Letaief, K.B., 2016. Dynamic computation offloading for mobile-edge computing with energy harvesting devices. *IEEE Journal on Selected Areas in Communications* 34, 3590–3605.
- [37] MYERSON, R.B., 1991. *Game Theory: Analysis of Conflict*. Harvard University Press. URL: <http://www.jstor.org/stable/j.ctvjsf522>.
- [38] Nesterov, Y., Nemirovskii, A., 1994. Interior-point polynomial algorithms in convex programming. *SIAM*.
- [39] NVIDIA, . Compare geforce rtx laptops. URL: <https://www.nvidia.com/en-us/geforce/laptops/compare/>.
- [40] Peng, Z., Liao, G., 2024. Nbs-based mobile edge computing: A joint optimization of qos and energy in computation offloading, in: *IFIP International Conference on Network and Parallel Computing*, Springer. pp. 40–50.
- [41] PPIO, . URL: <https://ppio.com/>.
- [42] Pu, X., Lei, T., Wen, W., Feng, W., Wang, Z., Chen, Q., Jin, S., 2023. Incentive mechanism and resource allocation for collaborative task offloading in energy-efficient mobile edge computing. *IEEE Transactions on Vehicular Technology* 72, 13775–13780. doi:10.1109/TVT.2023.3274513.
- [43] Ren, J., Liu, J., Zhang, Y., Li, Z., Lyu, F., Wang, Z., Zhang, Y., 2022. An efficient two-layer task offloading scheme for mec system with multiple services providers, in: *IEEE Conference on Computer Communications*, pp. 1519–1528.
- [44] RenderNetwork, . Maximize the productivity of gpu compute power. URL: <https://rendernetwork.com/participate-node-operators>.
- [45] Rezaeianfardouei, H., Townley, M., Saedi, M., 2025. Reducing Failover Latency in Cisco ASA Site-to-Site VPNs Through IPsec Parameter Tuning, in: *2025 IEEE Conference on Dependable, Autonomic and Secure Computing (DASC)*, IEEE Computer Society, Los Alamitos, CA, USA. pp. 108–114. URL: <https://doi.ieeeecomputersociety.org/10.1109/DASC68382.2025.00021>, doi:10.1109/DASC68382.2025.00021.
- [46] Saedi, M., Moore, A., Jaafar, F., Rahman, M.A., Biswas, S., 2026. Zero trust rule-based protocol for secure 5g handover against rogue base stations. *IEEE Open Journal of the Communications Society* 7, 5092–5106. doi:10.1109/OJCOMS.2026.3691214.
- [47] SaladCloud, . About salad cloud. URL: <https://salad.com/about>.
- [48] Shen, H., Tian, Y., Wang, T., Bai, G., 2024. Slicing-based task offloading in space-air-ground integrated vehicular networks. *IEEE Transactions on Mobile Computing* 23, 4009–4024. doi:10.1109/TMC.2023.3283852.
- [49] Shortle, J.F., Thompson, J.M., Gross, D., Harris, C.M., 2018. *Fundamentals of queueing theory*. John Wiley & Sons.
- [50] Theta, 2025. Theta edge node. URL: <https://docs.thetatoken.org/docs/setup-theta-edge-node>.
- [51] Tsigkari, D., Iosifidis, G., Spyropoulos, T., 2024. Quid pro quo in streaming services: Algorithms for cooperative recommendations. *IEEE Transactions on Mobile Computing* 23, 1753–1768. doi:10.1109/TMC.2023.3240006.
- [52] Wang, H., Huang, J., 2018. Incentivizing energy trading for interconnected microgrids. *IEEE Transactions on Smart Grid* 9, 2647–2657. doi:10.1109/TSG.2016.2614988.
- [53] Wang, S., Guo, Y., Zhang, N., Yang, P., Zhou, A., Shen, X., 2021a. Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach. *IEEE Transactions on Mobile Computing* 20, 939–953.
- [54] Wang, X., Ren, X., Qiu, C., Cao, Y., Taleb, T., Leung, V.C.M., 2021b. Net-in-ai: A computing-power networking framework with adaptability, flexibility, and profitability for ubiquitous ai. *IEEE Network* 35, 280–288. doi:10.1109/MNET.011.2000319.
- [55] Wang, Y., Tao, X., Zhang, X., Zhang, P., Hou, Y.T., 2019. Cooperative task offloading in three-tier mobile computing networks: An admm framework. *IEEE Transactions on Vehicular Technology* 68, 2763–2776.
- [56] Wen, Y., Zhang, W., Luo, H., 2012. Energy-optimal mobile application execution: Taming resource-poor mobile devices with cloud clones, in: *IEEE Conference on Computer Communications*, pp. 2716–2720. doi:10.1109/INFCOM.2012.6195685.
- [57] Wilkes, J., 2020. Google ClusterData 2019 Traces. <https://github.com/google/cluster-data/blob/master/ClusterData2019.md>. Accessed: 2026-07-20.
- [58] Wu, H., Wolter, K., 2018. Stochastic analysis of delayed mobile offloading in heterogeneous networks. *IEEE Transactions on Mobile Computing* 17, 461–474. doi:10.1109/TMC.2017.2711014.
- [59] Xie, N., Weber, S., 2017. On the aloha throughput-fairness tradeoff. *IEEE Transactions on Information Theory* 63, 6085–6112.
- [60] Yang, Y., Liu, Z., Liu, Z., Xie, Y., Chan, K.Y., Guan, X., 2022. Joint optimization of edge computing resource pricing and wireless caching for blockchain-driven networks. *IEEE Transactions on Vehicular Technology* 71, 6661–6670. doi:10.1109/TVT.2022.3162075.
- [61] Yi, C., Cai, J., Zhang, T., Zhu, K., Chen, B., Wu, Q., 2021. Workload re-allocation for edge computing with server collaboration: A cooperative queueing game approach. *IEEE Transactions on Mobile Computing* 22, 3095–3111.
- [62] Yin, L., Luo, J., Qiu, C., Wang, C., Qiao, Y., 2024. Joint task offloading and resources allocation for hybrid vehicle edge computing systems. *IEEE Transactions on Intelligent Transportation Systems* 25, 10355–10368.
- [63] Yu, H., Cheung, M.H., Huang, J., 2017. Cooperative wi-fi deployment: A one-to-many bargaining framework. *IEEE Transactions on Mobile Computing* 16, 1559–1572. doi:10.1109/TMC.2016.2599157.
- [64] Zhang, Y., Fan, Z., Hou, J., Chen, N., Lyu, F., Yang, P., 2025. Incentivizing point cloud-based accurate cooperative perception for connected vehicles. *IEEE Transactions on Vehicular Technology* 74, 5637–5648. doi:10.1109/TVT.2024.3519626.
- [65] Zhao, G., Xu, H., Zhao, Y., Qiao, C., Huang, L., 2020. Offloading dependent tasks in mobile edge computing with service caching, in: *IEEE Conference on Computer Communications*, pp. 1997–2006. doi:10.1109/INFOCOM41043.2020.9155396.
- [66] Zhou, H., Wu, T., Chen, X., He, S., Guo, D., Wu, J., 2023. Reverse auction-based computation offloading and resource allocation in mobile cloud-edge computing. *IEEE Transactions on Mobile Computing* 22, 6144–6159. doi:10.1109/TMC.2022.3189050.
- [67] Zhu, B., Chi, K., Liu, J., Yu, K., Mumtaz, S., 2022. Efficient offloading for minimizing task computation delay of noma-based multiaccess edge computing. *IEEE Transactions on Communications* 70, 3186–3203. doi:10.1109/TCOMM.2022.3162263.
- [68] Zilic, J., De Maio, V., Ilager, S., Brandic, I., 2025. Fresco: Fast and reliable edge offloading with reputation-based hybrid smart contracts. *IEEE Transactions on Services Computing* 18, 3810–3823. doi:10.1109/TSC.2025.3609833.



Beiyang Liu received the B.E. degree from Sun Yat-sen University in 2022 and the M.S. degree from the School of Software Engineering, Sun Yat-sen university in 2026. His research interests include edge computing, network economics, and game theory.



Yifei Wu received the B.E. degree from Central China Normal University in 2023. He is currently pursuing the Ph.D. degree with the School of Intelligent Systems Engineering, Sun Yat-sen University. His research interests include edge and cloud computing, network economics, and online learning.



Guocheng Liao received the Ph.D. degree with Department of Information Engineering, the Chinese University of Hong Kong in 2021, and received the B.E. degree from Sun Yat-sen University in 2016. He is now an Associate Professor with School of Software Engineering, Sun Yat-sen University. His current research lies in edge intelligence and federated learning. He received 2017 IEEE GLOBECOM ComSoc Young Professional Best Paper Award.



Jianguo Chen received his Ph.D. degree in Computer Science and Technology from Hunan University, China, in 2018. He was a visiting Ph.D. student at the University of Illinois at Chicago from 2017 to 2018. He is currently an Associate Professor and one of the Hundred Academic Talents in the School of Software Engineering of Sun Yat-sen University (SYSU), China. Before joining SYSU, he was a postdoc at the University of Toronto in Canada and a research scientist at the Agency for Science, Technology and Research (A*STAR) in Singapore. His major research interests include high-performance artificial intelligence, federated learning, distributed computing, and the application in intelligent transportation and intelligent medicine. He has published more than 70 research papers in international conferences and journals such as IEEE-TII, IEEE-TITS, IEEE-TPDS, IEEE-TKDE, IEEE/ACM-TCBB, ACM-TIST, and ACM-TCPS. He is currently serving as an Associate Editor in the International Journal of Embedded Systems and Journal of Current Scientific Research.



Qian Ma is an Associate Professor of School of Intelligent Systems Engineering, Sun Yat-sen University. She worked as a Postdoc Research Associate at Northeastern University during 2018-2019. She received the Ph.D. degree in the Department of Information Engineering from the Chinese University of Hong Kong in 2017, and the B.S. degree from Beijing University of Posts and Telecommunications (China) in 2012. Her research interests lie in the field of network optimization and economics. She is the recipient of the Best Paper Award from the IEEE International Symposium on Modeling and Optimization in Mobile, Ad Hoc and Wireless Networks (WiOpt) in 2021 and the recipient of the Best Student Paper Award from the IEEE International Symposium on Modeling and Optimization in Mobile, Ad Hoc and Wireless Networks (WiOpt) in 2015.



Xu Chen received the Ph.D. degree in information engineering from the Chinese University of Hong Kong in 2012. He is a Full Professor with Sun Yat-sen University, Guangzhou, China, and the Vice Director of the National and Local Joint Engineering Laboratory of Digital Home Interactive Applications. He was a Post-Doctoral Research Associate with Arizona State University, Tempe, USA, from 2012 to 2014, and a Humboldt Scholar Fellow with the Institute of Computer Science, University of Goettingen, Germany, from 2014 to 2016. He was a recipient of the Prestigious Humboldt Research Fellowship awarded by Alexander von Humboldt Foundation of Germany, the 2014 Hong Kong Young Scientist Runner-Up Award, the 2016 Thousand Talents Plan Award for Young Professionals of China, the 2017 IEEE Communication Society Asia-Pacific Outstanding Young Researcher Award, the 2017 IEEE ComSoc Young Professional Best Paper Award, the Honorable Mention Award of 2010 IEEE international conference on Intelligence and Security Informatics, the Best Paper Runner-Up Award of 2014 IEEE International Conference on Computer Communications (INFOCOM), and the Best Paper Award of 2017 IEEE International Conference on Communications. He is currently an Area Editor of IEEE Open Journal of the Communications Society, an Associate Editor of the IEEE Transactions Wireless Communications, IEEE Internet of Things Journal and IEEE Journal on Selected Areas in Communications (JSAC) Series on Network Softwarization and Enablers.